

Resumen del capítulo: Algoritmos de machine learning

Regresión lineal y funciones de error

La regresión es una técnica en el aprendizaje supervisado. Aquí el modelo debe identificar una relación entre las características de los objetos y el valor de la variable objetivo. Con la regresión, la variable objetivo es un número en vez de una clase.

Recordemos lo que se dio en la tarea:

n — el número de observaciones en el dataset

k — el número de características

$\vec{y}$ — el vector (lista) de valores de variables objetivo para nuestros datos

X — una matriz objeto-característica ($n \times k$) donde las filas son observaciones y las columnas son características

Dentro de la matriz, cada observación tiene un vector o conjunto de valores característica correspondiente:

$$\vec{x} = (x_1, x_2, \dots, x_k)$$

La matriz X se ve así:

$$X = \begin{pmatrix} x_{1_1} & \cdots & x_{1_k} \\ \vdots & \ddots & \vdots \\ x_{n_1} & \cdots & x_{n_k} \end{pmatrix}$$

Estamos buscando una fórmula única que hará un pronóstico (estimaré), sobre cualquier objeto, utilizando los valores de características k :

$$f(\vec{x}) = f(x_1, x_2, \dots, x_k) = \hat{y}$$

Si encontramos tal fórmula y la aplicamos a todos los objetos del *dataset*, conseguiremos un vector de pronóstico (set) para todos los objetos:

$$\vec{\hat{y}} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n) - \text{un vector de pronóstico (lista)}$$

Donde:

$\hat{y}_i$ – n pronóstico para una determinada observación i

es

El vector resultante se puede comparar con el vector original de la variable objetivo para descubrir cuán buena es la fórmula. Esto se puede hacer, por ejemplo, calculando el error promedio para todas las observaciones. Primero pensaremos cómo podría verse la fórmula, después ajustaremos sus parámetros. La forma más sencilla es asumir que la variable objetivo se describe con una función lineal de características:

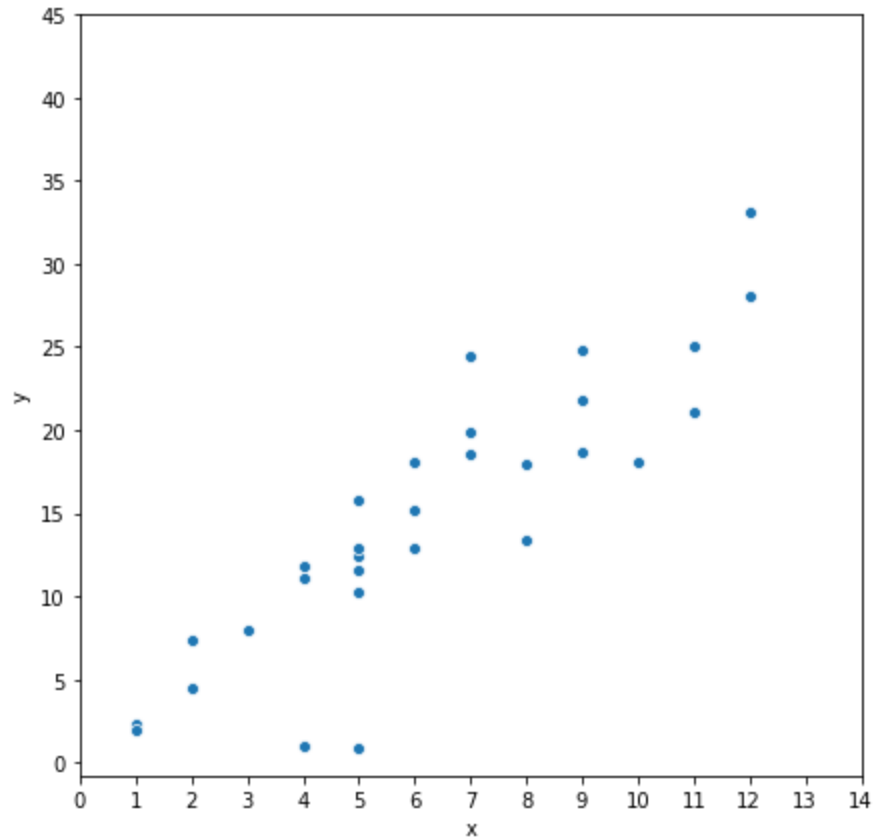
$$y(x) = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_k * x_k$$

Aquí:

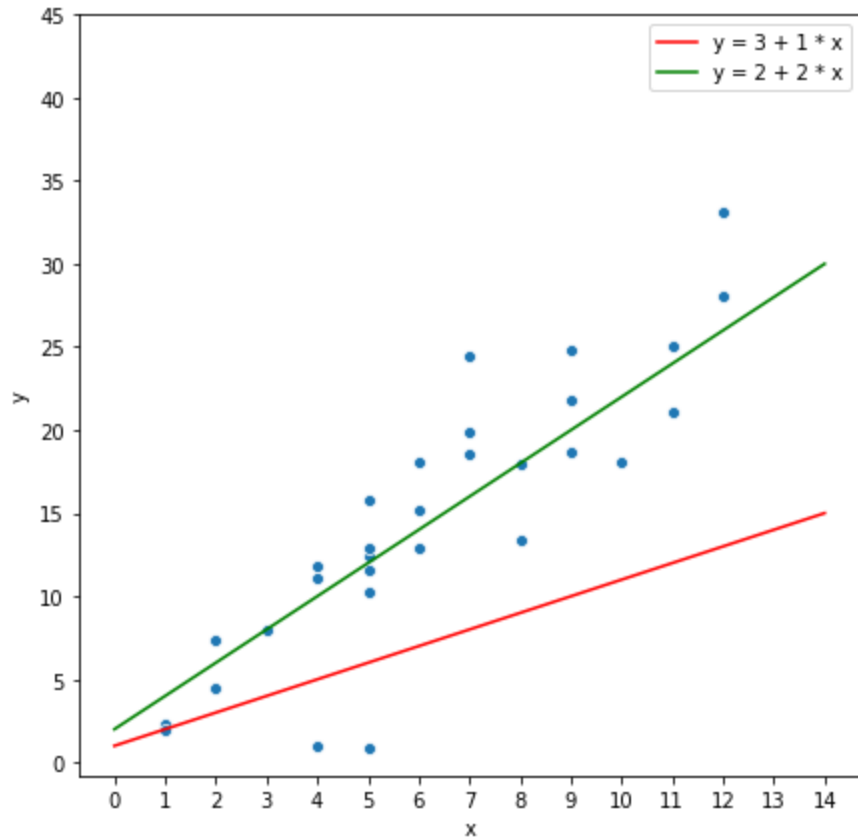
$(w_0, w_1, w_2, \dots, w_k)$ – coeficientes numéricos

Estos coeficientes se llaman **pesos**: determinan la contribución de cada característica para la suma que define nuestro pronóstico. Así que necesitamos encontrar los valores de peso que habilitarán la función para producir resultados que serán muy cercanos a los valores reales.

Asumamos que tenemos una sola característica para cada observación. Trazaremos un gráfico donde cada observación tenga un punto, con la característica a lo largo del eje X y el valor de la variable objetivo a lo largo del eje Y:



Necesitamos encontrar una función $y = w_0 + w * x$ que producirá un gráfico lineal y esa línea debe describir bien nuestras observaciones. Algunas líneas caracterizan un conjunto o **data cloud** (nube de datos) mejor que otras. Por ejemplo, puedes trazar un gráfico para la función $y = 3 + 1 * x$ o $y = 2 + 2 * x$:

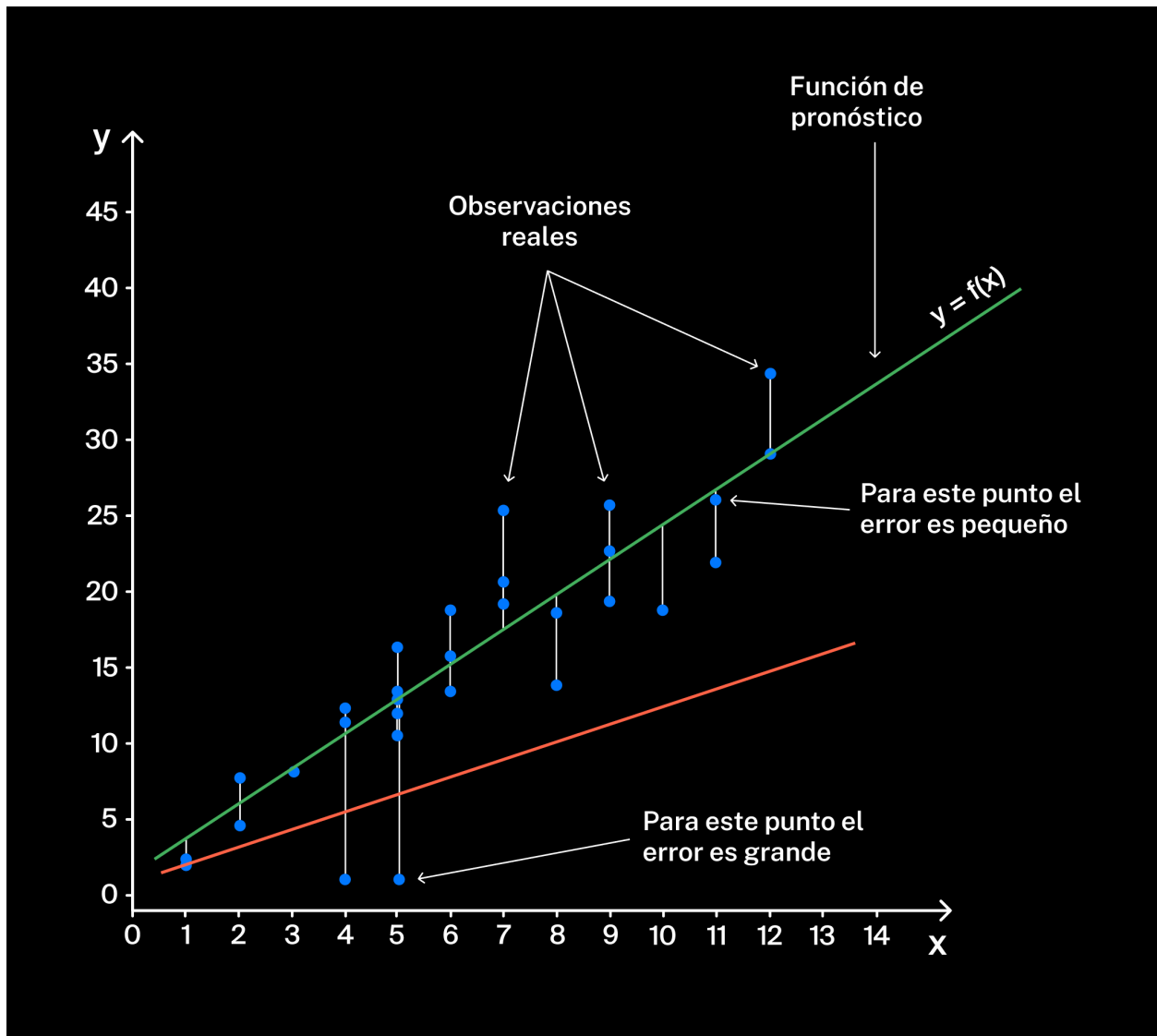


La línea verde describe las observaciones mejor que la roja.

Pero, ¿cómo podemos describir más formalmente hasta qué punto una función refleja la interrelación real entre las características?

La forma más sencilla es estimar **qué tanto fallamos** en nuestros pronósticos en comparación con el valor real de la variable objetivo.

Introduzcamos la **función de error** $Q(w)$. Dependerá de los pesos que hemos elegido.



¿Qué deberíamos considerar un error? La diferencia entre el pronóstico y los valores reales, ¿no? De hecho, eso no es del todo cierto. La diferencia entre la expectativa y la realidad puede ser negativa, pero el error no. Si el pronóstico es perfecto, el error es 0. Si un valor pronosticado es menor o mayor que el valor real, el error será positivo.

Para asegurar que el error es una valor positivo, lo elevamos al cuadrado:

$$(y_i - \hat{y}_i)^2$$

Con ciertos objetos el modelo errará más que con otros. Para estimar el error general, necesitamos encontrar el error medio para todos los objetos:

$$Q(w) = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

Hemos definido la función de error. Ahora tenemos que encontrar pesos (w) que minimizarán el valor de la función de error.

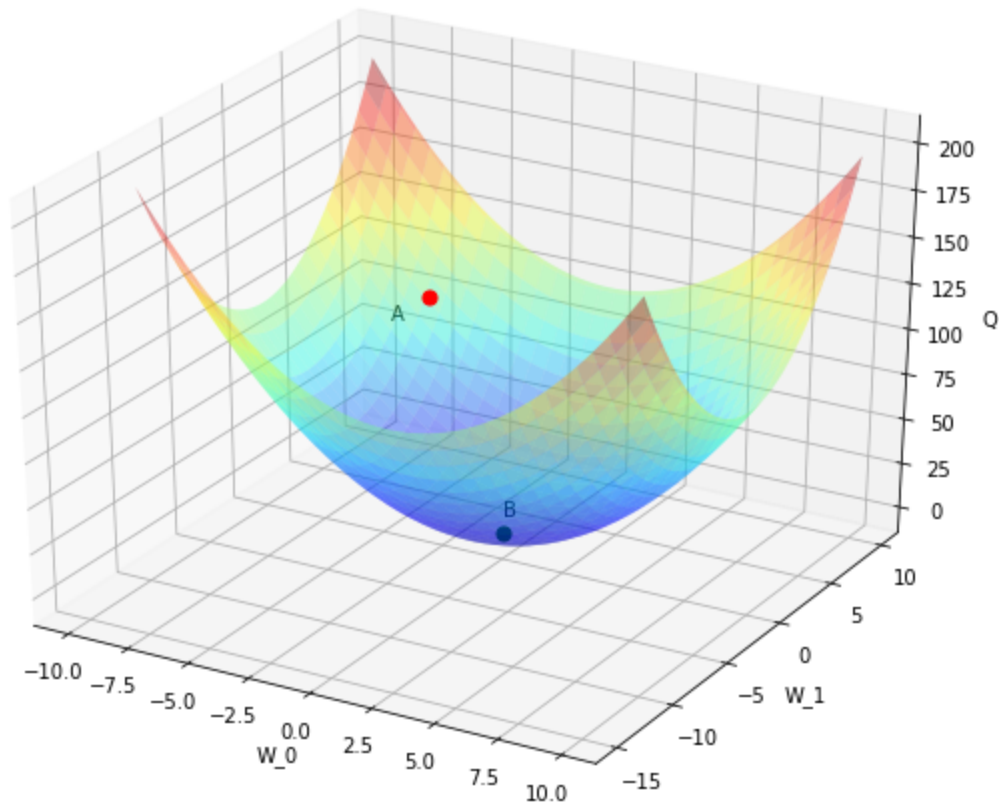
Descenso de gradiente

El error que comete un modelo con un conjunto de pesos dado se define por la fórmula:

$$Q(w) = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

donde $y(x) = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_k * x_k$

Tu tarea es encontrar la combinación de pesos que minimizarán este valor. Volvamos al caso donde teníamos solo una característica y dos coeficientes para los que necesitamos elegir valores: $y = w_0 + w_1 * x$. Podemos calcular los valores de la función de error para diversos valores w_0 y w_1 . Entonces el gráfico de la función de error se verá así:



Esto es un **gráfico tri-surface** (tres superficies). Vamos a suponer que hemos elegido los valores de los dos coeficientes w_{0_a} y w_{1_a} y conseguido el valor de la función Q_a . En la imagen, esto corresponde con el punto A. Necesitamos encontrar el punto donde el valor de la función será el más pequeño. We need to find the point where the value of the function will be the smallest. Lo marcaremos como B. El punto B corresponde con el conjunto de pesos para los que el error medio cuadrático de las observaciones es el más pequeño. Si pensamos de nuevo en el ejemplo de la lección anterior, podemos decir que el punto A corresponde con la línea roja y el punto B con la verde.

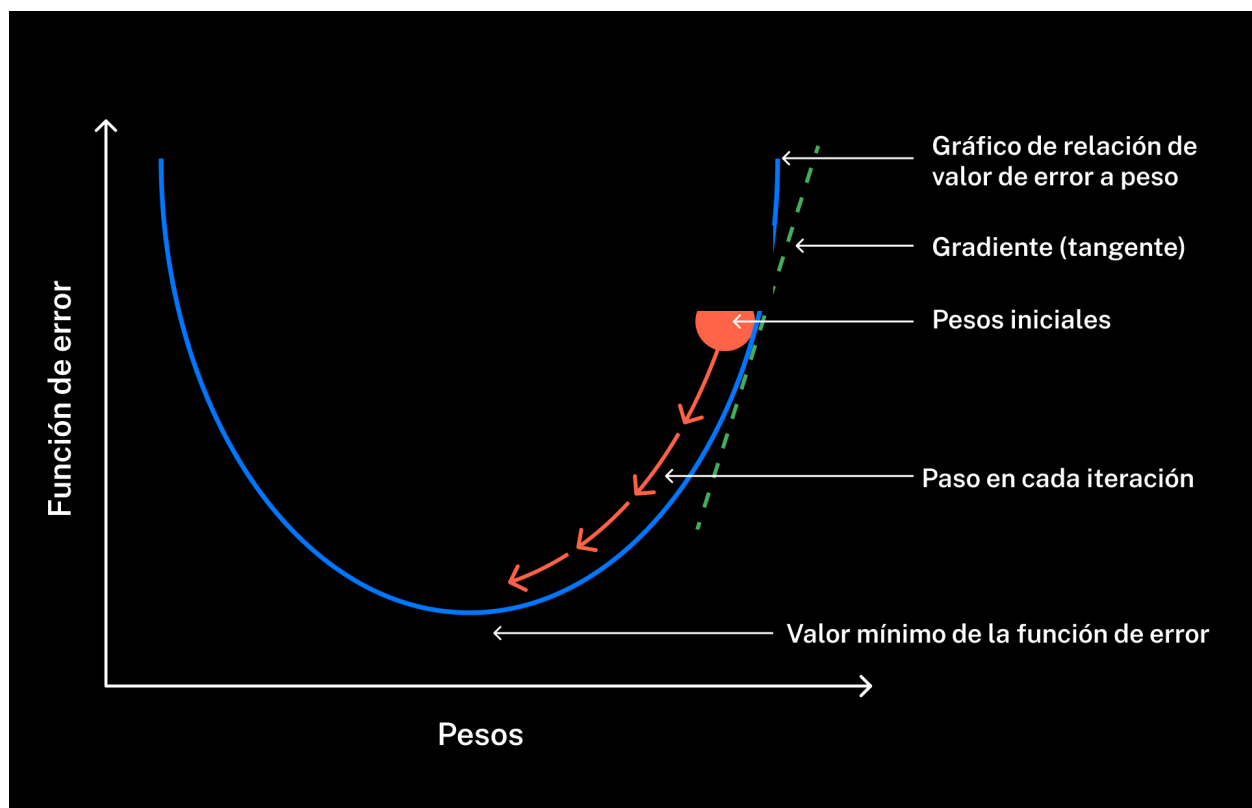
La búsqueda para el mínimo local es el quid de la optimización. En el aprendizaje automático se aplica un método de optimización especial: **el descenso de gradiente**. Elegimos un punto inicial (en nuestro caso, punto A) con valores de peso iniciales. Se desplaza paso a paso hacia el mínimo de la función de error.

Las preguntas cruciales en este método son:

- ¿En qué dirección deberíamos movernos?

- ¿Cuán largo debería ser el paso?

¿Dónde exactamente necesitamos movernos? Tendría sentido movernos en la dirección en la que el descenso al mínimo es el más inclinado. Esto significa que necesitamos movernos en la dirección que da el mayor descenso en el valor de la función para un paso de un ancho dado. Si estamos tratando con dos dimensiones, el desplazamiento es a lo largo de la tangente desde el punto actual hacia abajo. Si arreglamos el valor del coeficiente cero w_0 , entonces el segmento de la superficie que describe la dependencia del error Q en el peso w_1 dada una característica se verá como una curva parabólica:



Nos movemos desde el punto inicial hacia abajo a lo largo de la tangente —si has estudiado cálculo, sabrás que esta es la derivada de la función. En este caso, nos movemos a lo largo de la derivada de la función de error con respecto a los pesos (el eje X):

$$\frac{\partial Q}{\partial w_1}$$

Si tenemos varias dimensiones, la tangente a la superficie es el vector de las derivadas para cada peso:

$$\nabla Q(w) = \left(\frac{\partial Q}{\partial w_0}, \dots, \frac{\partial Q}{\partial w_k} \right)$$

Este vector se llama **gradiente**. Es positivo. Necesitamos movernos hacia abajo hacia el mínimo de la función. Esto significa que nos moveremos en la dirección opuesta, hacia el **gradiente negativo**.

Hemos averiguado la dirección. Ahora tenemos que averiguar cómo movernos rápido: qué tan largos deben ser los pasos para llegar al "suelo". De esto es responsable la **tasa de aprendizaje**. La llamaremos α . Después podemos formular la corrección del peso en cada paso así:

$$\vec{w}_{t+1} = \vec{w}_t - \alpha \nabla Q(\vec{w}_t)$$

El paso no puede ser muy largo o podríamos saltarnos el mínimo. Pero tampoco puede ser muy pequeño o el entrenamiento tomará mucho tiempo. Normalmente, se elige un **hiperparámetro** en los experimentos. Este término sirve para distinguirlo de los parámetros de función w . El valor del hiperparámetro inicial es relativamente pequeño, por ejemplo, 0.001 o 0.003. Aquí el paso (es decir, el hiperparámetro) es la **velocidad de aprendizaje**.

En la práctica, el descenso de gradiente básico rara vez se utiliza, principalmente debido a las siguientes desventajas:

- Si la superficie de la función tiene demasiadas "cavidades", dependiendo de la selección de peso inicial podría acabar en un **mínimo local** en lugar del **mínimo global** de la función. Así que tenemos que ejecutar el proceso varias veces, cambiando las suposiciones iniciales.
- Calcular el gradiente para todas las muestras toma mucho tiempo. Para acelerar la convergencia e intentar prevenir el sobreajuste, se aplica el descenso de **gradiente aleatorio (estocástico)**. Aquí el gradiente se calcula para subconjuntos o **mini-batches** en lugar de para un conjunto entero.
- La superficie de la función de error puede ser compleja. En tales casos, no solo aplicamos direcciones, sino también derivadas adicionales de la función de error.

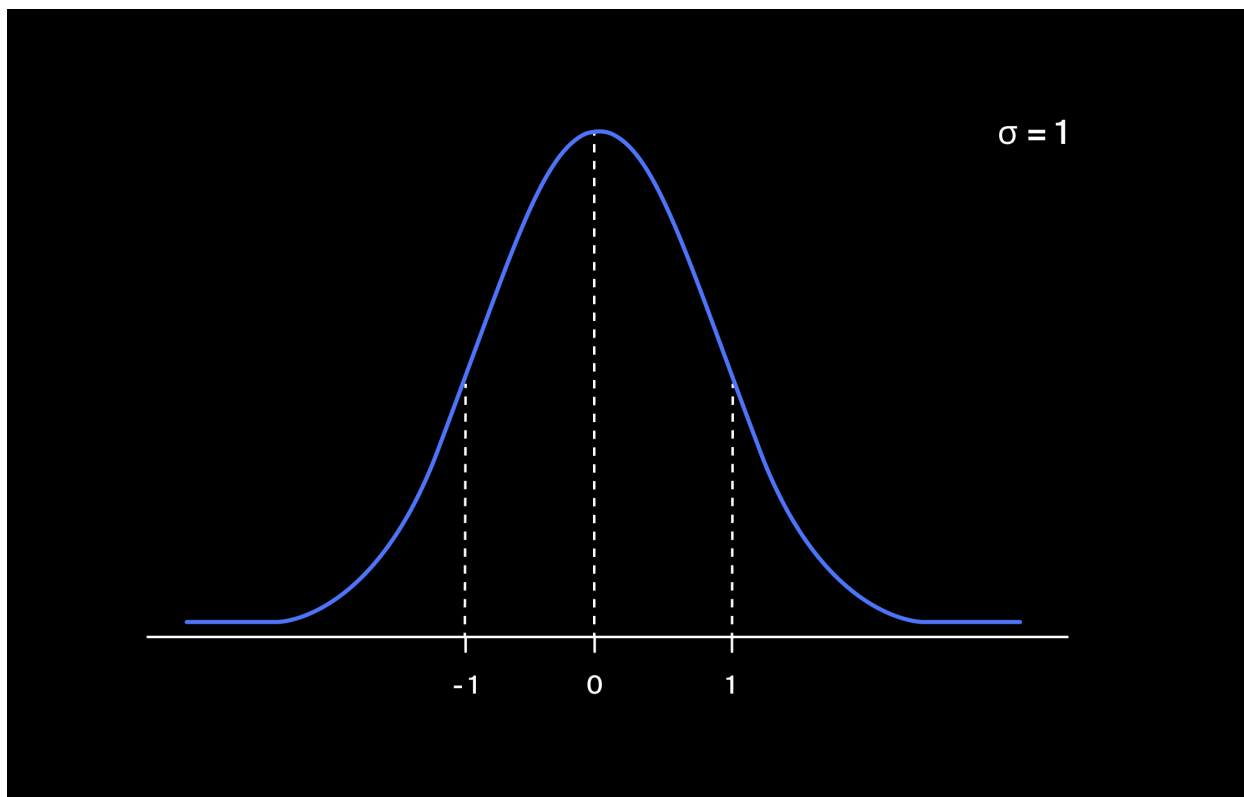
Preprocesamiento: escalar características

Cuando estás llevando a cabo una regresión lineal, ten dos cosas en cuenta:

- Las características deberían ser estandarizadas.
- El modelo de regresión lineal es propenso al sobreajuste y es inestable cuando las características se correlacionan mutuamente.

Los rangos de valores de las características pueden diferir significativamente. Si empiezas seleccionando coeficientes, verás que los pesos para las diferentes características deben considerar la escala de esas características. Las características de menor escala deben tener pesos más grandes y las otras deben tener pesos más pequeños. Al mismo tiempo, los valores de los coeficientes deben permitir estimar la contribución de cada característica a la suma ponderada global.

Para comparar los pesos, debemos unificar la escala de varias características. Esto puede hacerse mediante la **normalización**, que implica convertir los valores de característica al rango de 0 a 1. Sin embargo, el escalado se lleva a cabo típicamente a través de la **estandarización**, lo que significa que hacemos que la distribución tenga una media de 0 y una desviación estándar de 1. (¿Te suena? Esa es la **distribución normal estándar**). La razón de esto es que solo una vez que tenemos tal distribución tiene sentido minimizar el error cuadrático medio.



La estandarización transforma un valor de característica para que se vea así: para cada observación, el valor de característica inicial disminuye por la media y la diferencia resultante se divide entre la desviación estándar. En la práctica, no necesariamente tienes que saber la fórmula por la cual a las características iniciales se les asignan valores de estos rangos. No obstante, para resolver tareas con métodos de regresión lineal (por ejemplo, esto también es cierto para el *clustering*, que discutiremos más tarde), los datos deben ser estandarizados.

El módulo `preprocessing` en `sklearn` tiene las clases `MinMaxScaler()` y `StandardScaler()` preparadas para normalizar y estandarizar datos, respectivamente. Parecen modelos: primero necesitan ser entrenadas o que les muestren qué valores toma una característica en los datos de entrenamiento. Solo entonces pueden aplicarse a las nuevas observaciones. La normalización y la estandarización tienen la misma sintaxis:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler() # crear un objeto de clase scaler
```

```
scaler.fit(X) # entrenar estandarizador  
X_sc = scaler.transform(X) # transformar el dataset
```

En línea con el *pipeline* de aprendizaje automático estándar, divide los datos en conjuntos de entrenamiento y validación. Para estandarizar características, tienes que saber su media y desviación estándar para las observaciones, pero no puedes predecir los valores de característica. Deberías tener en cuenta este hecho cuando valides.

Ya que un modelo solo ve la media y la desviación estándar mientras está entrenando sobre datos de entrenamiento, la estandarización se hace en los siguientes pasos:

- Calcula la media y la desviación estándar en los datos de entrenamiento (entrenando `StandardScaler()`) y transforma los datos.
- Aplica el `StandardScaler()` entrenado en los datos de validación (tomando en cuenta la media y la desviación estándar de los datos de entrenamiento).
- Entrena el modelo en los datos de entrenamiento estandarizados.
- Haz un pronóstico para los datos estandarizados en el conjunto de validación.

Regularización

Los modelos lineales simples son fáciles de crear e interpretar, pero puede haber algún problema con ellos. Esto puede ser debido a la **multicolinealidad**, por ejemplo.

La multicolinealidad ocurre cuando tenemos un grupo de características que están linealmente relacionadas (es decir, interdependientes o que tienen fuertes correlaciones). A lo mejor dos características que corresponden a la misma distancia se han medido sigilosamente en el *dataset*. Una está en metros y la otra está en kilómetros. O las características podrían estar fuertemente relacionadas, como la temperatura del agua o la temperatura del aire. Si el coeficiente de correlación es demasiado alto (generalmente más alto que 0.8), puede que tengas problemas con la regresión lineal: **pesos no representativos y un modelo sobreajustado para estas características**. Pero, ¿por qué?

Aquí está un ejemplo ilustrativo. Tenemos dos características idénticas, el caso definitivo de la alta correlación mutua:

$$x_1 = x_2$$

Y ahora observa estas tres ecuaciones:

$$\begin{aligned} y &= w_0 + w_1 * x_1 + w_2 * x_2 \\ y &= w_0 + (w_1 + w_2) * x_1 + 0 * x_2 \\ y &= w_0 + 100000 * (w_1 + w_2) * x_1 - 99999 * (w_1 + w_2) * x_2 \end{aligned}$$

¡Todas tienen el mismo resultado! Los errores de los pronósticos de estos modelos de acuerdo a los valores reales también serán los mismos. El algoritmo no puede decidir cuál de los tres vectores es mejor. Puede haber un número infinito de tales vectores equivalentes. Si el algoritmo los utiliza, fallará al dar una estimación adecuada del impacto que tiene una característica individual en una variable objetivo.

¿Cómo manejamos esto? Manteniendo solo aquellas características cuyas correlaciones están por debajo del umbral (0.8). Por ejemplo, podemos calcular una matriz de correlación utilizando el método `corr()` y, cuando sea que encontremos un par de características fuertemente correlacionadas, eliminar una de ellas manualmente:

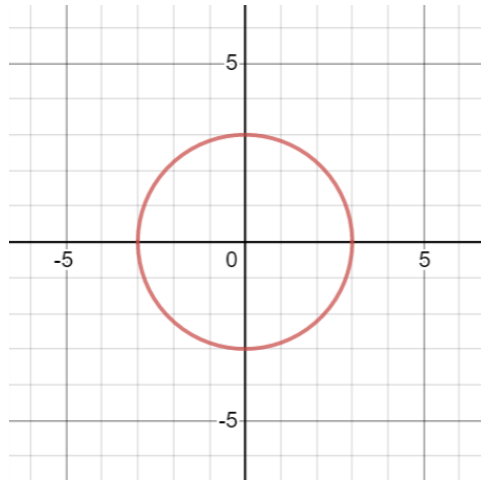
```
cm = df.corr() # calcular la matriz de correlación
```

Si no quieres eliminar características correlacionadas a mano, puedes llevar a cabo la **regularización**. Generalmente hablando, esto se refiere a cualquier limitación adicional en un modelo o a una acción que disminuya la complejidad del modelo y minimice el impacto del sobreajuste. En el caso de modelos lineales, la regularización implica un límite en los pesos.

Podemos establecer la condición directamente de la siguiente forma:

$$\sum_{i=1}^m w_i^2 < b$$

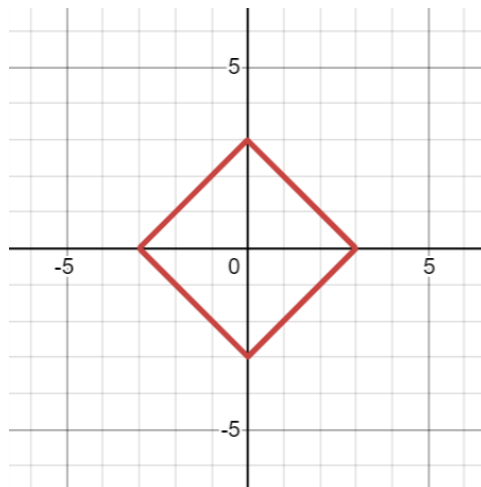
Una limitación similar para dos características y `b=3` se vería así:



Otra forma de limitar la escala de pesos es limitar su valor absoluto:

$$\sum_{i=1}^m |w_i| < a$$

Una limitación similar para dos características y $a=3$ se vería así:



Tales limitaciones a menudo se establecen directamente mediante la función de error. De esta forma, tratamos de minimizar el error con los pesos de característica más pequeños (en términos de la suma de los valores absolutos o la suma de cuadrados). En otras palabras, prohibimos que el modelo elija pesos de características altos de

modo que no se calibre a sí mismo en función de las características de multicolinealidad. Podemos crear limitaciones de dos formas.

La **regularización L1** o **regresión Lasso** se asocia a esta fórmula:

$$Q_{Lasso} = \frac{1}{n} * \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda * ||w||_1 \right)$$

donde Q_{Lasso} es la función de error,
 y_i es la variable objetivo real para el objeto i ,
 $\hat{y}_i$ es el pronóstico para el objeto i ,
 λ es el coeficiente de regularización, y
 $||w||_1$ es la suma total de los valores absolutos de los pesos

(Es decir, cuando minimizamos el error al elegir los pesos óptimos, también tratamos de minimizar la **suma de los valores absolutos** de esos pesos.)

Regularización L2 o **regresión Ridge**:

$$Q_{Ridge} = \frac{1}{n} * \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda * ||w||_2^2 \right)$$

donde Q_{Ridge} es la función de error,
 y_i es la variable objetivo real para el objeto i ,
 $\hat{y}_i$ es el pronóstico para el objeto i ,
 λ es el coeficiente de regularización, y
 $||w||_2^2$ es la suma total de los pesos al cuadrado

(Esto es que mientras minimizamos el error eligiendo los pesos óptimos, también tratamos de minimizar la **suma de los cuadrados** de estos pesos).

`sklearn` tiene varios algoritmos de regresión lineal que ya incluyen regularización.

- La **regresión Lasso** utiliza la regularización L1. Este algoritmo pone a cero todos los pesos de características fuertemente correlacionadas, excepto por uno. Por consiguiente, este algoritmo tiene un mecanismo integrado para la **selección de**

característica. Se utiliza cuando necesitamos disminuir la dimensionalidad y deshacernos de las características duplicadas.

- La **regresión Ridge** utiliza la regularización L2. En este caso, los pesos entre las características correlacionadas se distribuyen casi equitativamente.

Implementar modelos lineales

La librería `sklearn` tiene un módulo `linear_model` que contiene algoritmos de regresión lineal.

- `linear_model.LinearRegression()` : el modelo para la regresión lineal estándar. Antes de aplicarlo, necesitas normalizar los datos y filtrar las características a mano para evitar la multicolinealidad.
- `linear_model.Lasso()` : un modelo de regresión lineal con regularización de peso L1 incorporada (limitando la suma total de valores absolutos de los pesos).

Esto también requiere datos normalizados, pero aquí la regularización integrada hace todo el filtrado por ti. No necesitas deshacerte de las características fuertemente correlacionadas por tu cuenta.

- `linear_model.Ridge()` : un modelo de regresión lineal con regularización de peso L2 integrada (limitando la suma total de pesos cuadrados).

Aquí también tienes que normalizar los datos, ya que las escalas de las características deben estar unificadas. Tanto con la regresión Ridge como con Lasso, no necesitas deshacerte de las características correlacionadas. La regresión Ridge por sí sola no filtrará características por ti, puesto que no pone a cero los valores de las características duplicadas. La función de regularización se diseña para distribuir los pesos equitativamente entre características similares.

Puedes estudiar la sintaxis de cada una de las funciones en la documentación oficial (en inglés):

https://scikit-learn.org/stable/modules/classes.html#module-sklearn.linear_model

Con las tareas de regresión lineal, entrenamos un modelo y realizamos predicciones utilizando los métodos `fit()` y `predict()` estándar. Por ejemplo, para la regresión Ridge, el código se ve así:


```
from sklearn.linear_model import Ridge

model = Ridge() # crear un modelo de la clase de regresión Ridge
model.fit(X_train, y_train)
y_pred = model.predict(X_val)
```

Los algoritmos de regresión lineal son diferentes en el sentido en que nos permiten interpretar pesos para cada característica. Tras la estandarización, los valores de los coeficientes de las características reflejan el impacto que cada uno de ellos tiene en la predicción final. Cuanto mayor sea el valor absoluto del coeficiente, mayor será su impacto. Habiendo entrenado el modelo, serás capaz de generar directamente todos los coeficientes, que corresponden a la función final óptima. Los pesos de las características se muestran por medio del método `.coef_` del modelo:

```
print(model.coef_)
```

y el valor del coeficiente cero, o coeficiente lineal, se muestra con el método

```
.intercept_.
```

```
print(model.intercept_)
```

Métricas de regresión

Las métricas se estudian en los datos de validación para comprobar si el modelo funciona bien en datos que no ha visto en el entrenamiento. Esto es lo que sucede en situaciones de la vida real: aplicas un modelo cuando no se conocen las respuestas. Todas las fórmulas para calcular métricas aquí dadas son para datos de validación.

El objetivo principal de una tarea de regresión es acercarse lo máximo posible al valor real.

Tenemos:

y_i como la variable objetivo real para la observación i ,
 $\hat{y}_i$ como el pronóstico para la observación i , y
 n como el número de observaciones en el set de validación.

Echemos un vistazo a las métricas más populares en negocio:

- Error absoluto medio, **EAM**
- Error cuadrático medio, **ECM**, y su raíz, **RECM**
- Coeficiente de determinación o R-squared (**R2**)
- Error absoluto medio porcentual, **MAPE**

Error absoluto medio (EAM)

El nombre de esta métrica revela cómo se calcula. Primero, estimamos la medida en la que nuestros pronósticos se desvían de los valores reales de una variable objetivo (utilizando valores absolutos). Luego, encontramos el valor absoluto del error medio para todas las observaciones en el conjunto de validación. Tomamos el valor absoluto de la diferencia, así que no importa si la desviación es positiva o negativa:

$$EAM = \frac{1}{n} * \sum_{i=1}^n |y_i - \hat{y}_i|$$

Error cuadrático medio (ECM) y su raíz (RECM)

También podemos hacer que la métrica refleje la sensibilidad del modelo a los valores atípicos. Si el modelo tiene errores más grandes para algunos valores, tenemos que hacer que esto amplifique este error aún más. Entonces necesitaremos encontrar el error cuadrático medio:

$$MSE = \frac{1}{n} * \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

... o su raíz:

$$RECM = \sqrt{\frac{1}{n} * \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Coeficiente de determinación o R cuadrado (R^2)

Ya conociste esta métrica, que muestra la proporción de la variación de la variable objetivo que tiene en cuenta el modelo. La fórmula del coeficiente de determinación es:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (\bar{y} - y_i)^2}$$

donde:

$$\bar{y} = \frac{1}{n} * \sum_{i=1}^n y_i$$

El coeficiente de determinación toma valores de 0 a 1. Cuanto más cercano sea a 1, más preciso será tu pronóstico para igualar los valores reales de la variable objetivo.

Error absoluto medio porcentual (MAPE)

Conocer el error como un porcentaje puede ser más importante que conocerlo como un valor absoluto (como EAM, ECM y RECM). Digamos que `MAE=42`, ¿eso es mucho o poco? Si la media de tu valor es 56, una desviación de 42 sugeriría que tu modelo no funciona. Pero si la variable objetivo oscila entre 3000 y 4000, 42 es un resultado muy bueno (sospechosamente bueno). El error absoluto medio porcentual se encuentra con la fórmula:

$$MAPE = 100\% * \frac{1}{n} * \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|}$$

MAPE es más claro intuitivamente. Un error absoluto medio porcentual del 2% inspira optimismo. Sin embargo, si es el 95%, debes volver a trabajar en tu modelo, buscar algún error en tus cálculos o incluso encontrar nuevos datos. Pero aún así, esta sensación de incertidumbre no desaparece: en algunos casos, una precisión del 80% no es suficiente, pero en otros, incluso el 95% será demasiado poco.

Métricas y la función de error

Cuando buscamos la función óptima, todos los algoritmos de las librerías estándar de Python minimizan la función de pérdida a medida que seleccionan los pesos. Esta

función a menudo coincide con la métrica que estás estimando en el set de validación. Por ejemplo, muchos algoritmos toman ECM como la función de pérdida por defecto. Pero a veces la función predeterminada y la métrica deseada no coinciden. Entonces el algoritmo "ajustará" las cosas que no necesitas.

En casi todas las implementaciones de algoritmo, puedes especificar la función a ser optimizada justo en la etapa de entrenamiento. Si quieres comparar la función de pérdida con la métrica que planeas estimar con los datos de validación, lee la documentación.

Calcular métricas en sklearn

Los desarrolladores de `sklearn` crearon el módulo `metrics` que te permite calcular varias métricas:

- EAM: `metrics.mean_absolute_error`
- ECM: `metrics.mean_squared_error`
- R²: `metrics.r2_score`

Las métricas RECM y MAPE no son parte de este módulo, pero tú puedes escribir las funciones para ellas si es necesario.

Estudiemos la sintaxis para trabajar con métodos métricos observando esta función

`metrics.mean_absolute_error`:

```
from sklearn.metrics import mean_absolute_error

print('MAE:', mean_absolute_error(y_true, y_pred))
```

Pasamos el vector de valores verdaderos de la variable objetivo (`y_true`) y el vector de valores predichos (`y_pred`) como parámetros. Las métricas `metrics.mean_squared_error` y `metrics.r2_score` se calculan de forma análoga.

Los datos de entrada reales dados al modelo tras ser desarrollado y entrenado se llaman **datos de prueba**. Sin embargo, antes de probar el modelo en situaciones reales, los y las analistas lo ajustan en los **datos de validación**.

Regresión logística

Este es un algoritmo para resolver tareas de clasificación binaria. Recuerda que la clasificación binaria es un caso de clasificación donde hay solo dos clases: "0" o "1". Aquí, la variable objetivo se denomina **variable binaria**.

Los modelos entrenados con algoritmos de clasificación binaria no solo pueden predecir el valor final de una clase para un cierto objeto o cliente, sino también estimar la probabilidad de un evento en cuestión.

Un algoritmo común para resolver tales tareas es la **regresión logística**. Se implementa como la clase `LogisticRegression()` dentro del módulo `linear_model` de `sklearn`, junto con un algoritmo de regresión lineal:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
```

Veamos la teoría tras este algoritmo. Tu función debe devolver la probabilidad de que un cierto evento ocurra. Asumamos que se encuentra con la siguiente fórmula:

$$f(z) = \mathbb{P}\{y = 1 \mid x\}$$

Esta ecuación significa que la probabilidad de que y sea igual a 1, dado que las características toman los valores x , se determina por la función f aplicada a la variable z .

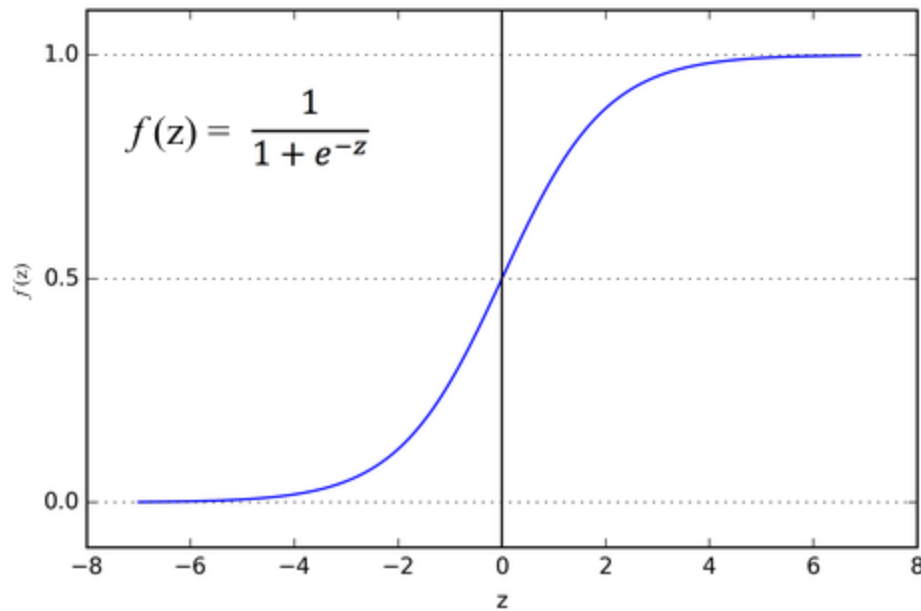
Aquí z es una función lineal de los valores característica (el vector x) como en una regresión lineal:

$$z(x) = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n$$

En la regresión logística, asumimos que f es una **función logística**. Se ve así:

$$f(z) = \frac{1}{1 + e^{-z(x)}}$$

El gráfico de la función se ve así:



La gráfica de la función logística se llama una **curva sigmoide**. El gráfico claramente muestra que la función toma los valores de 0 a 1. La forma sigmoide y el rango de tolerancia son realmente buenos para representar la probabilidad de ocurrencia de algunos eventos. Esta línea se "curva" en valores z infinitamente grandes e infinitamente pequeños. Si la suma ponderada tiene valores altos, para los cuales la probabilidad de ocurrencia de eventos es, digamos, 0.95, los aumentos adicionales en estas características no harán que esta alta probabilidad sea significativamente mayor. A medida que las características incrementan, se acerca a 1. Para probabilidades pequeñas, se acerca a 0.

Entonces un algoritmo (por ejemplo, para el descenso de gradiente) selecciona los coeficientes w para la función $z(x)$, de modo que $f(z)$ pueda volver a aceptar respuestas más cercanas a los valores reales de la variable objetivo.

De hecho, cuando estamos trabajando en una tarea de clasificación binaria utilizando regresión logística, transformamos la regresión lineal para encontrar la probabilidad de que un evento entre en la clase 1. Es por eso que las regresiones logísticas y lineales (y sus variaciones de Lasso y Ridge) pertenecen a los modelos lineales. Se pueden encontrar en el mismo módulo `linear_model` de la librería `sklearn`.

Para hacer una predicción para una clase tras entrenar tal modelo, necesitarás el método `predict()`:

```
y_pred = model.predict(X_test)
```

Y para obtener la probabilidad de que un objeto entre la primera o la segunda clase, utiliza el método `predict_proba()`:

```
y_probas = model.predict_proba(X_test)
```

`y_probas` es una matriz bidimensional donde cada objeto del conjunto de validación tiene dos valores correspondientes: la probabilidad (o más bien la confianza) de pertenecer a la clase 0 ("el evento no sucederá") y la probabilidad de pertenecer a la clase 1 ("el evento sucederá"). La suma de estos dos valores es igual a 1 para cada uno de los objetos.

Métricas de clasificación: trabajar con etiquetas

Métricas de clasificación basadas en clases predichas

Empecemos con las métricas que tienen en cuenta solo el valor predicho final (0 o 1):

- **Matriz de confusión**
- **Exactitud**
- **Precisión y *recall***
- **Puntuación F1**

Matriz de confusión

Los valores de clase posibles son 0 y 1 y tu modelo puede generar una predicción final en la forma de uno de esos valores. Por consiguiente, la predicción para cada objeto entra en uno de los cuatro grupos:

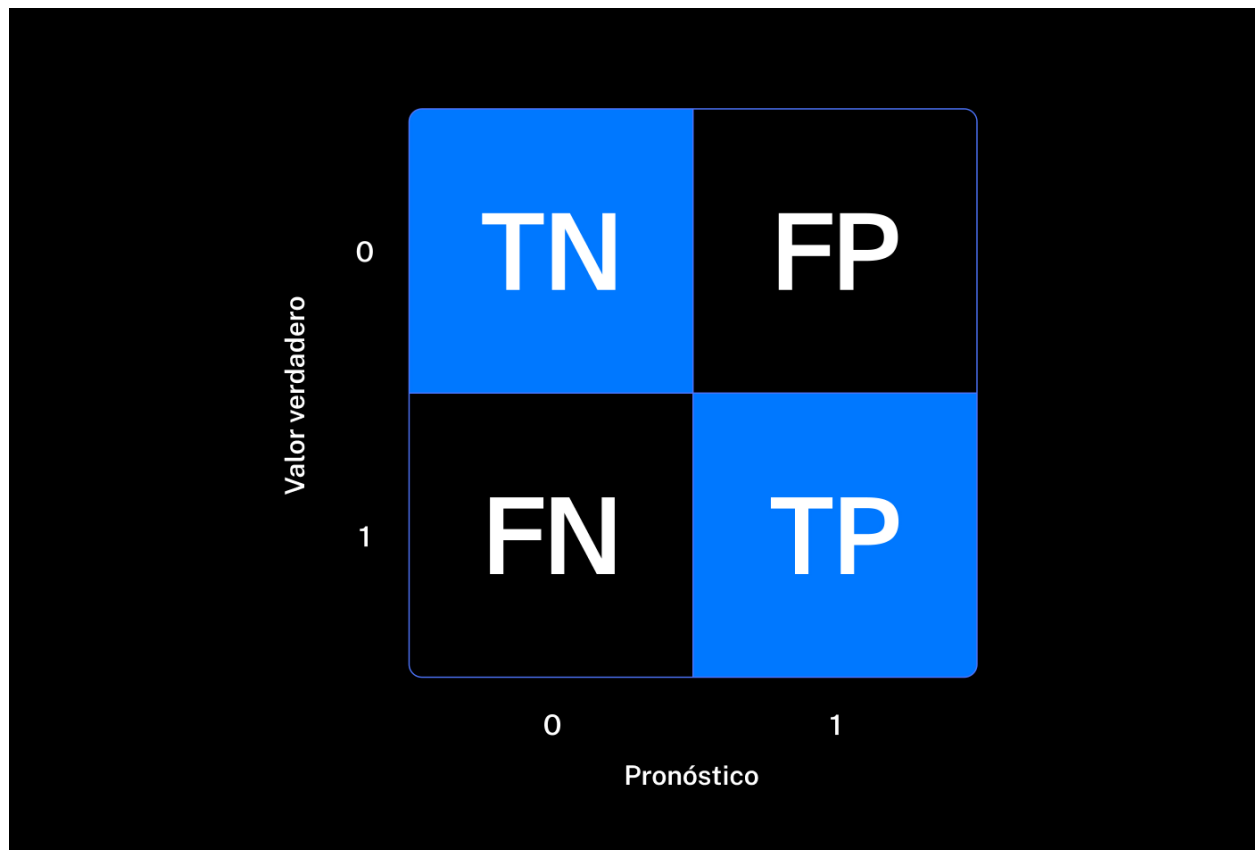
- Predicción del modelo = 1, valor real = 1. Tales predicciones se llaman **verdaderos positivos** o **TP** (del inglés '*True Positive*').
- Predicción del modelo = 1, valor real = 0. Tales predicciones se llaman **falsos positivos** o **FP** (del inglés '*False Positive*').

- Predicción del modelo = 0, valor real = 1. Tales predicciones se llaman **falsos negativos** o **FN** (del inglés *'False Negative'*).
- Predicción del modelo = 0, valor real = 0. Tales predicciones se llaman **verdaderos negativos** o **TN** (del inglés *'True Negative'*).

Si un modelo es bueno, la mayoría de sus predicciones entran en los grupos TP y TN.

Una matriz de confusión muestra el número de observaciones para cada grupo.

Debería tener este aspecto:



La matriz de confusión se calcula en el módulo `metrics` de sklearn. También puedes definir las variables TN, FP, FN y TP:

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_true, y_pred)
tn, fp, fn, tp = cm.ravel() # "igualar" la matriz para conseguir
```


La función `confusion_matrix()` devuelve una matriz cuadrada igual al número de clases. El método `.ravel()` nos permite transformar la matriz en una tupla pivotante (unidimensional) `[TN, FP, FN, TP]`.

Se puede ver a partir de la matriz de confusión qué errores tiende a cometer tu algoritmo. A lo mejor tiene un sesgo hacia la clase positiva (optimismo excesivo, demasiados FP) o podría ser pesimista y expandir el grupo FN para asegurarse.

Los términos que hemos introducido para la matriz de confusión son ciertos para el resto de métricas de pronóstico de clases.

Exactitud

La exactitud se puede encontrar así:

$$Exactitud = \frac{TP + TN}{n} = \frac{TP + TN}{TP + TN + FP + FN}$$

Esta es la proporción de predicciones exactas entre todas las predicciones. Cuanto más cerca estemos del 100% de exactitud, mejor.

Esta métrica puede calcularse con la función `accuracy_score` en el módulo `metrics`. La función toma valores de clase actuales y predichos para los datos de validación como *input*:

```
acc = accuracy_score(y_true, y_pred)
```

La exactitud solo funciona cuando las **clases están equilibradas**, es decir, cuando los objetos se distribuyen casi equitativamente entre las clases, aproximadamente 50/50.

Precisión y recall

Para evaluar un modelo sin mirar el equilibrio de clases, puedes calcular las métricas:

$$Precisión = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

La precisión nos dice qué proporción de predicciones en la clase `1` es verdadero. En otras palabras, miramos la **proporción de respuestas correctas solo en la clase objetivo**. En los negocios, esta métrica se necesita solo cuando cada **alerta** del modelo (un caso en el que un objeto se coloca en la clase `1`) cuesta recursos. Y tú obviamente no quieres que el modelo alerte sin un buen motivo.

La segunda métrica aspira a minimizar los riesgos opuestos. *Recall* muestra **el número de objetos de clase `1` reales que fuiste capaz de descubrir** con tu modelo.

Cada métrica toma valores de 0 a 1. Cuanto más cerca de 1, mejor. Sin embargo, cuando estás ajustando los parámetros del modelo (normalmente el umbral de probabilidad más allá del cual colocamos un objeto de la clase `1`), optimizar una métrica puede a menudo llevar al deterioro de la otra. Por esa razón, tienes que buscar el equilibrio entre esas dos métricas a medida que ajustas tu modelo. El producto final debería basarse en los objetivos de tu trabajo.

Las métricas de precisión y *recall* se generan mediante las funciones `precision_score` y `recall_score` en el módulo `metrics`. Su sintaxis es similar a la de otras métricas:

```
precision = precision_score (y_true, y_pred)
recall = recall_score (y_true, y_pred)
```

Ambas funciones devuelven valores numéricos desde 0 a 1.

Puntuación F1

Puesto que la precisión y *recall* tienen como objetivo evitar riesgos opuestos, necesitamos una métrica armonizadora que tenga en cuenta el equilibrio entre las métricas. Esta métrica es la **puntuación F1**:

$$F1 = \frac{2 * \textit{precisión} * \textit{recall}}{\textit{precisión} + \textit{recall}}$$

En `sklearn.metrics` la puntuación F1 se encuentra a través del método `f1_score`:

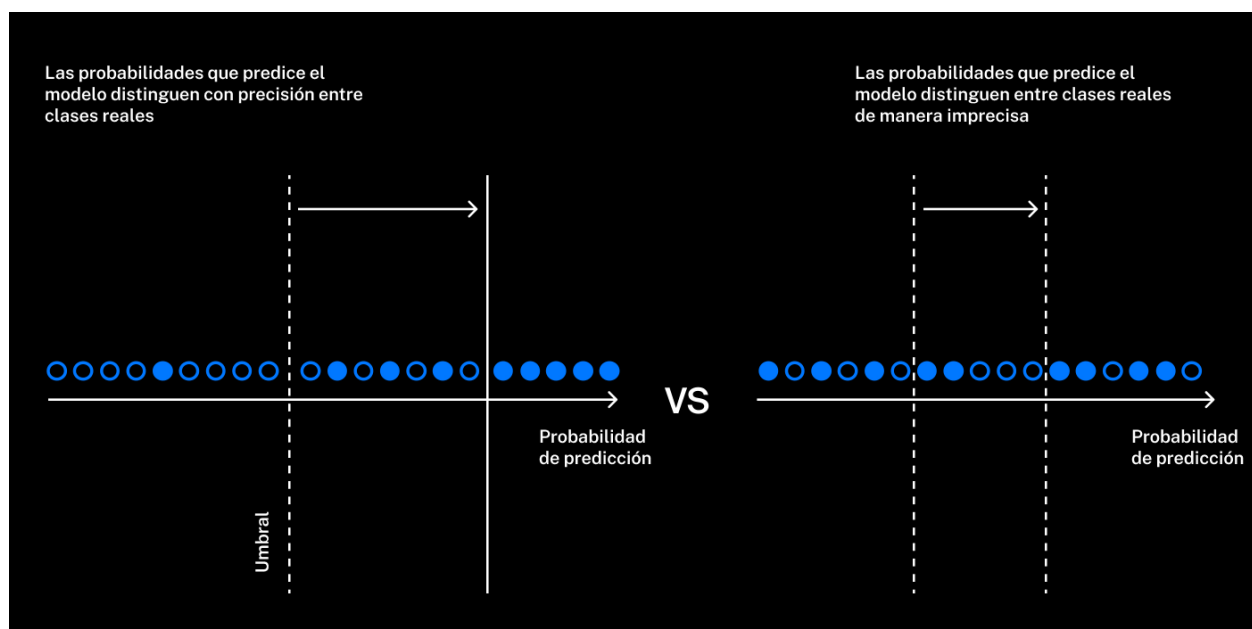
```
f1= f1_score(y_true, y_pred)
```

La función también devuelve un valor entre 0 y 1. Cuanto más cerca de 1, mejor.

Métricas de clasificación: estudio de probabilidades

Además de la respuesta para cada observación (0 o 1), también podemos calcular la probabilidad de que una observación pertenezca a una de las clases (0 ó 1). Es por eso que usamos la puntuación `roc_auc`, o el **área bajo la curva de característica operativa del receptor**, para evaluar la calidad de un clasificador (un modelo de clasificación). **AUC-ROC** son las siglas de área bajo la curva (*Area Under Curve*) y característica operativa del receptor (*Receiver Operating Characteristic*).

Esta métrica tiene la siguiente lógica: "Si ordenamos los objetos según la probabilidad predicha por el modelo, ¿veremos una distinción clara entre las clases reales de estos objetos?" En otras palabras, ¿los objetos de la clase 0 estarán concentrados en un extremo y los objetos de la clase 1 en el otro? Si las probabilidades ordenan mal nuestros objetos, otras métricas no mejorarán, sin importar el umbral que seleccionemos. Pero cuando los objetos están bien ordenados, podemos concluir que el modelo funciona adecuadamente y luego establecer el umbral.



La función de AUC-ROC hace de esta métrica una gran herramienta, incluso para trabajar con clases desequilibradas.

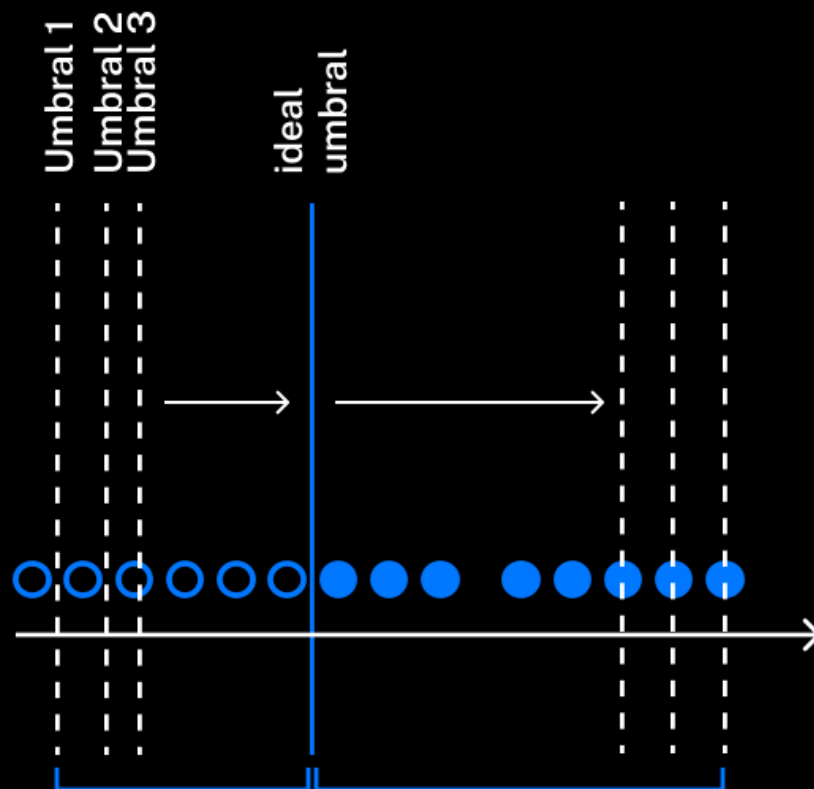
Echemos un vistazo más de cerca a cómo funciona. Vamos a presentar los términos **tasa de verdaderos positivos** (TVP corresponde a sensibilidad) y **tasa de falsos**

positivos (TFP es para aquellos objetos que el algoritmo colocó incorrectamente en la clase 1 en lugar de 0):

$$TVP = \frac{TP}{TP + FN}$$

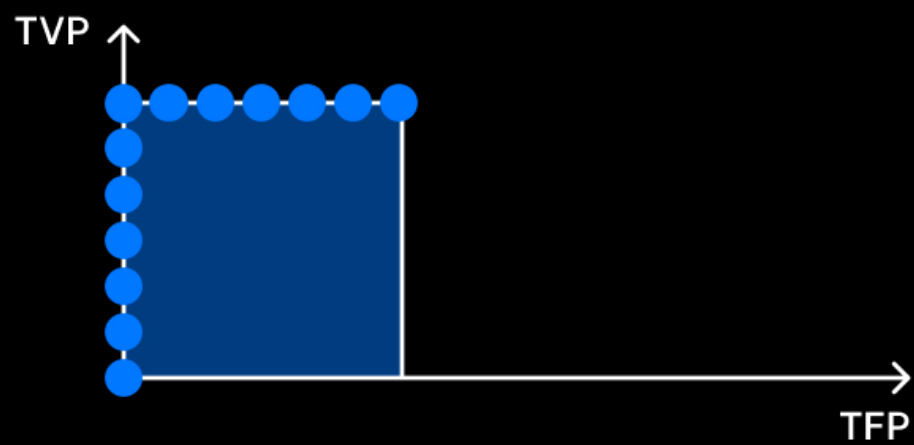
$$TFP = \frac{FP}{FP + TN}$$

Vamos a ordenar los objetos en orden ascendente según la probabilidad predicha y seleccionaremos de forma iterativa el umbral para la asignación en clases. Nos detendremos en cada observación para calcular la TVP y la TFP para el umbral determinado. Así se verían los objetos en un modelo perfecto:

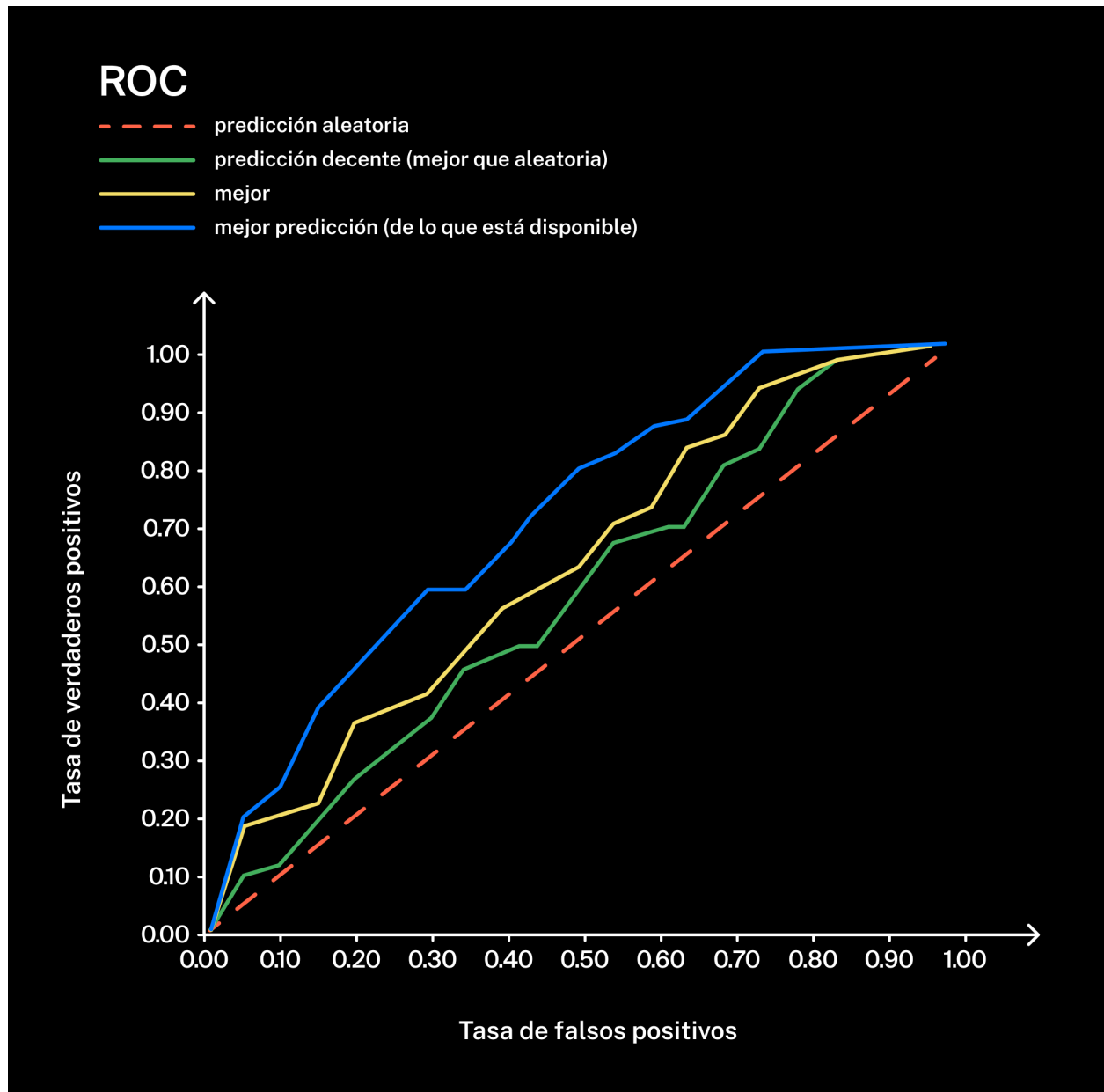


$TP = 1$
 $FN = 1$
 $TVP = 0$
 $TFP = 0$

$TVP = 1$
 $TFP = 1$



Pero en la vida real, ni los datos ni los modelos son perfectos. Nunca seremos capaces de lograr una ordenación perfecta por probabilidad. Entonces, en la mayoría de los casos, la curva ROC se ve así:



Cuanto más alto se curve la línea en la dirección de la esquina superior izquierda, es decir, cuanto más se acerque a 1, mejor. Cuanto más cerca esté de la línea **TVP=TFP** (de la esquina inferior izquierda a la esquina superior derecha), peor. Esto se corresponde con el patrón de modelo "aleatorio", cuando tienes que adivinar la clase.

La métrica `roc_auc` se puede calcular en el módulo `metrics` y se llama `roc_auc_score`:

```
roc_auc = roc_auc_score(y_true, probabilities[:,1])
```

El *input* de esta función es el vector de respuestas verdaderas `y_true` y el vector de probabilidad de asignación de clase `1`. Esto se puede hacer tomando solo los segundos números (`probabilities[:,1]`) del vector de probabilidades pareadas de asignar las clases `0` o `1`. La función genera un valor numérico único entre 0 (más a menudo 0.5) y 1.

Umbrales y balance de clases

Mientras selecciona los mejores pesos posibles para la función $z(x)$ por medio de la función sigmoide $f(z)$, el algoritmo de regresión logística calcula la probabilidad de que ocurra un evento. Luego calcula la respuesta final, `0` o `1`, según esta probabilidad. Esto se hace mediante la comparación con un determinado **umbral**. En la mayoría de los casos, el umbral es 0.5. Pero, ¿por qué?

De hecho, para muchos algoritmos, el método `predict_proba()` no muestra la probabilidad de pertenecer a una clase, sino el grado de certeza del modelo de que el objeto pertenece a la clase. Si la certeza del modelo sobre la pertenencia del objeto a la clase `1` es superior a 0.5, entonces la respuesta es `1`; de lo contrario, es `0`. Vamos a ver en qué casos la probabilidad y la "certeza" se pueden considerar lo mismo y en qué se diferencian.

Imagina que la mitad de los objetos del conjunto de entrenamiento pertenecen a la clase `0` y la otra mitad a la clase `1`. Estos conjuntos se llaman **equilibrados**.

Sin algoritmos ni conocimiento del nuevo objeto, no se puede decir con certeza a qué clase pertenece. Es un 50/50, ya sea `0` o `1`, ya que la probabilidad básica inicial de que un objeto pertenezca a cualquiera de las clases es 0.5. Entonces, esta probabilidad puede considerarse el grado de confianza de que un objeto pertenece a `1` o a `0`. Si puedes decir que la probabilidad de pertenecer a la clase `1` es mayor que 0.5 según las características que contienen información adicional sobre el objeto, entonces la certeza de que el objeto pertenece a la clase `1` es mayor que la de la clase `0`. Esto explica el umbral: asignamos `1` a todos los objetos para los cuales `predict_proba()` para esa clase produce un resultado superior a 0.5.

Mientras aprendes los conceptos básicos, completarás tareas de clasificación binaria simplificadas que no siempre son realistas. En estas tareas, los objetos se distribuyen de manera más o menos uniforme entre las clases. (Imagina una bolsa que contiene el mismo número de bolas blancas y negras). En la vida real, las clases rara vez están equilibradas, por lo que `predict_proba()` te da una probabilidad que se corrige en función del equilibrio entre las clases.

Son las clases desequilibradas las que revelan problemas con las métricas basadas en respuestas binarias y hacen imposible interpretar el resultado de `predict_proba()` como la probabilidad de pertenecer a una clase. Sin embargo, muchos algoritmos normalizan las probabilidades y aplican otros métodos matemáticos de tal forma que `predict_proba()` refleja la confianza del modelo en la respuesta binaria final. Al igual que antes, puedes establecer el umbral para el nivel de confianza al definir clases binarias. Al hacerlo, estarás corrigiendo la métrica que consideras más importante.

Árbol de decisión

Mientras que antes asumíamos que buscábamos una respuesta, el valor de la variable objetivo como la suma de valores de características ponderadas (una función lineal), los árboles de decisión usan **escenarios** para seleccionar una respuesta.

Ventajas de los árboles de decisión:

- Son fáciles de interpretar. En los negocios, este es a menudo el factor clave. Los algoritmos más complejos son como "cajas negras" porque sus procesos de toma de decisiones están ocultos.
- Trabajan tanto con clasificación como con regresión. En el último caso, las hojas del árbol tendrán números en lugar de valores de clase.
- Son bastante rápidos.
- No requieren un preprocesamiento de datos considerable: los árboles no son sensibles a la escala de las características ni son vulnerables a la multicolinealidad.

Desventajas:

- Son fuertemente propensos al sobreajuste. Recuerda, esto significa que el modelo adapta sus parámetros o la estructura de las ramas a los datos de entrenamiento. Funcionará muy bien en el conjunto de entrenamiento, pero fallará cuando se le

proporcionen nuevos datos. A veces, cuando hay muchas características, puedes crear una rama con un solo objeto o con unos pocos. El árbol se adaptará a los datos de entrenamiento e intentará colocar objetos de una sola clase en cada hoja.

Para evitarlo, **podamos** los árboles. La poda consiste en cortar una rama en un punto determinado para evitar el sobreajuste del árbol. Las muestras pequeñas son especialmente propensas al sobreajuste.

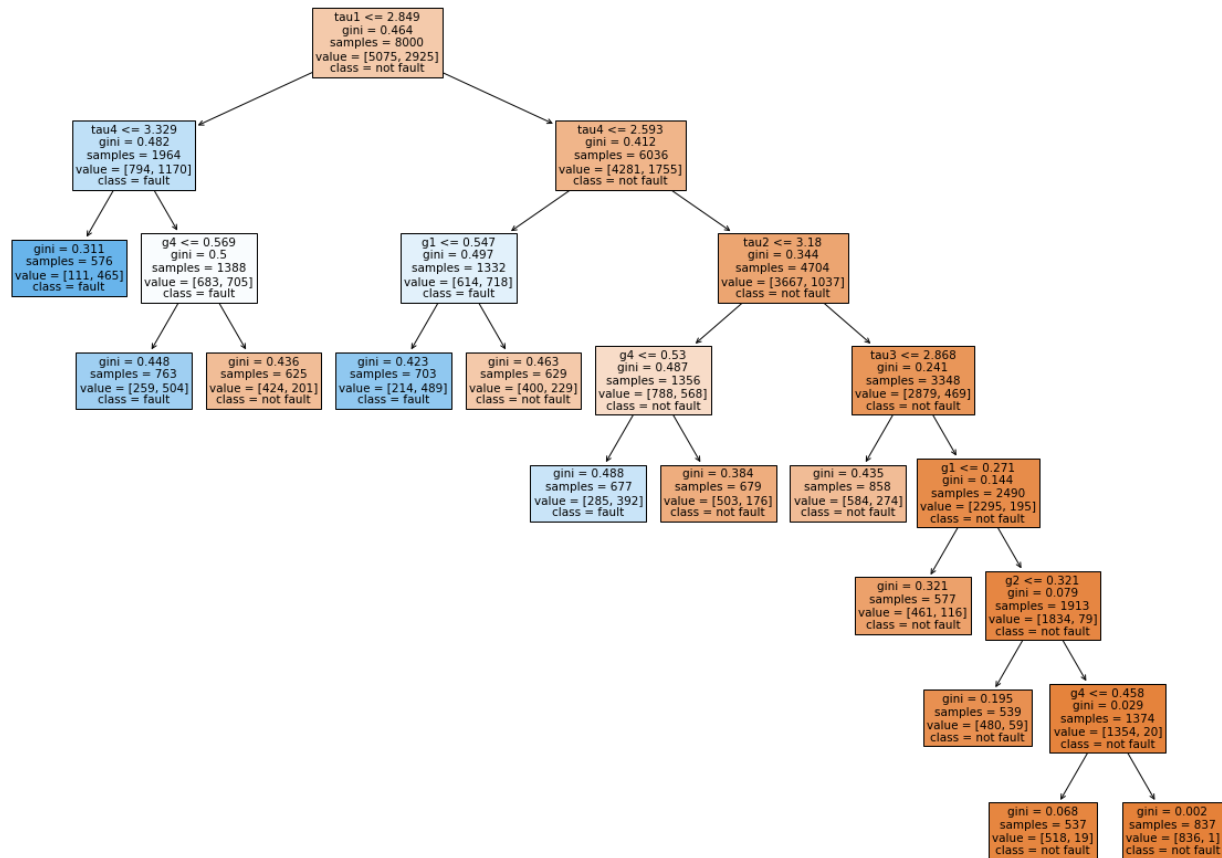
`sklearn` tiene un módulo `tree` que incluye los algoritmos `DecisionTreeClassifier` y `DecisionTreeRegressor`:

```
tree_model = DecisionTreeClassifier()  
tree_model.fit(X_train, y_train)  
y_pred = tree_model.fit(X_test)
```

En algunos casos, tendrás que visualizar el árbol de decisión resultante. Puedes usar la función **`plot_tree()`** del mismo módulo:

```
plt.figure(figsize = (20,15)) # configura el tamaño de la figura  
plot_tree(tree_model, filled=True, feature_names = X_train.columns)  
plt.show()
```

El resultado será algo así:



Este es el árbol del modelo `tree_model = DecisionTreeClassifier(min_samples_leaf=500)`, que resuelve una tarea de clasificación binaria ("fault" / "not fault"). La muestra contiene alrededor de 10 000 objetos. Cada nodo tiene una condición para la construcción de una rama adicional, así como el resultado más probable en la muestra correspondiente a este nodo. Ten cuidado: si la muestra es grande, establece el límite en el número mínimo de objetos en la hoja, por ejemplo, `min_samples_leaf=500`. Si ignoras esto, el árbol terminará con demasiadas ramas para una visualización útil.

Ensamblados de árboles de decisión: bosque aleatorio (*random forest*) y potenciación del gradiente (*gradient boosting*)

Los **ensambles** son potentes modelos de aprendizaje automático. Son poderosos y ayudan a descubrir relaciones complejas dentro de los datos. Pero el costo de este

poder es una capacidad de interpretar reducida. Estos algoritmos a veces se llaman **cajas negras**, ya que es difícil saber cómo una característica particular impactó la predicción de un objeto. Depende de ti decidir lo que te atreverías a sacrificar al resolver tareas de negocios.

Es mejor tener muchos árboles que solo uno

La idea básica de un ensamble es sencilla. Supongamos que has resuelto un problema de clasificación binaria con el modelo `m_1` basado en árboles de decisión. Has obtenido el valor `roc_auc_score_1 = 0.8`. Sin embargo, todos los modelos son propensos al sobreajuste, por lo que un algoritmo que funciona bien para las observaciones y características del conjunto de entrenamiento puede cometer grandes errores con los datos de validación. Podemos crear otro modelo, `m_2`, que compensará los errores del primer modelo. Por sí mismo, podría producir un `roc_auc` relativamente bajo (por ejemplo, `roc_auc_score_2=0.79`). Entonces puedes cambiar las condiciones una vez más y construir otro modelo, `m_3`, y así hasta `m_n`. Cada uno de los `n` modelos será diferente, sensible a factores diferentes y transmitirá información única sobre las observaciones. Puedes intentar considerarlos todos al mismo tiempo y hacer un modelo consolidado, `M`, que producirá mejores resultados que cada modelo tomado por separado (por ejemplo, `roc_auc_score = 0.91`). Se dice que dicho modelo es **fuerte** ("un clasificador fuerte") mientras que los modelos en los que se basa son **débiles** ("clasificadores débiles").

Hay dos tipos básicos de modelos de ensambles: **bosque aleatorio** y **potenciación del gradiente**. Con ellos, los árboles de decisión generalmente se consideran como modelos de predicción débiles. La selección de árboles no es aleatoria. A diferencia de los modelos lineales, por ejemplo, los árboles de decisión te permiten cambiar la observación y los subconjuntos de características, así como otros parámetros, para cada clasificador débil por separado. Por ejemplo, puedes limitar la profundidad del árbol o el número mínimo de objetos en cada nodo. De esta manera, los árboles proporcionan muchos más modelos para resolver una tarea determinada.

El promedio funciona mejor cuando los modelos difieren y compensan los errores de los demás. Pero si promedias modelos parecidos con defectos similares, no obtendrás ese efecto.

Bosques aleatorios

El primer tipo de ensamble se llama **bosque aleatorio**. Genera muchos **árboles mutuamente independientes** de formas ligeramente diferentes (tomando diferentes subconjuntos o características) y llega a una decisión final basada en sus respuestas. El algoritmo de bosque aleatorio **promedia las respuestas** de todos los árboles (en regresión) o **utiliza votación** (en clasificación) para seleccionar la respuesta que es verdadera según la mayoría de los árboles.

Al hablar de ensambles de modelos, podemos usar el término **bagging** (de "agregación de bootstrap"). Esto implica promediar modelos entrenados en diferentes subconjuntos. El enfoque en sí, que consiste en estimar varias características o predicciones utilizando una multitud de subconjuntos diferentes obtenidos del conjunto inicial, se llama **bootstrapping**.

Cuando trabajas con bosques aleatorios, recuerda que los árboles se entrenan en paralelo o de forma independiente. El entrenamiento de cada árbol no depende de los resultados de los demás. Esto representa una diferencia conceptual entre bosques aleatorios y potenciación del gradiente.

Potenciación del gradiente

El concepto básico es más o menos el mismo: recopilamos muchos modelos simples (árboles) para que compensen mutuamente los errores y la aleatoriedad. Solo que en este caso los errores se compensan a través de **entrenamiento**, en lugar de promediar.

En la potenciación de gradiente, los árboles se entrenan **consecutivamente**: cada árbol se construye con referencia a los resultados del anterior. Ya conoces con los principios del descenso de gradiente: tratamos de minimizar el error en cada paso a medida que avanzamos en la dirección que queremos. La idea detrás de la potenciación de gradiente es muy similar. En un paso determinado tenemos un modelo que hace un error en una predicción. Luego construimos otro modelo para predecir el error del primer modelo en lugar del valor original y ajustamos la predicción final sobre esta base. Luego, un tercer modelo predice el error del segundo modelo, y así sucesivamente. Finalmente, usamos la secuencia completa para hacer una predicción para una observación específica. Decimos que el primer modelo produjo este resultado pero le añadimos la corrección del segundo modelo, luego del tercero, y así hasta el último modelo. Así, con cada paso (cada nuevo modelo), mejoramos nuestras

predicciones anteriores. "Potenciación" (*boosting*) se refiere a este proceso de mejora y la potenciación del "gradiente" significa que la mejora tiene lugar paso a paso.

Uso de ensambles

En `sklearn`, los bosques aleatorios se implementan en el módulo de `ensemble`. Los algoritmos más conocidos del módulo son `RandomForestClassifier()` y `RandomForestRegressor()`. Los necesitamos para resolver problemas de clasificación y regresión, respectivamente.

Mira la sintaxis para declarar, entrenar y predecir en un problema de regresión:

```
from sklearn.ensemble import RandomForestRegressor

# definir el algoritmo del nuevo modelo basándose en el algoritmo
rf_model = RandomForestRegressor(n_estimators = 100)
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_val)
```

Cuando declaramos nuestro modelo, definimos `n_estimators`, el número de árboles en nuestro bosque. Para los parámetros restantes del árbol (su profundidad `max_depth`, el tamaño del subconjunto de características `max_features` y la cantidad mínima de objetos por nodo `min_samples_leaf`), dejamos los valores predeterminados sin cambios. Puedes jugar con estos valores y ver cómo afectan el resultado.

La potenciación del gradiente se implementa en el módulo `ensemble` de `sklearn`. Cuenta con los algoritmos `GradientBoostingClassifier()` y `GradientBoostingRegressor()` para resolver problemas de clasificación y regresión, respectivamente.

Mira su sintaxis en un problema de regresión:

```
from sklearn.ensemble import GradientBoostingRegressor

# definir el algoritmo del nuevo modelo basándose en el algoritmo
gb_model = GradientBoostingRegressor(n_estimators = 100)
gb_model.fit(X_train, y_train)
y_pred = gb_model.predict(X_val)
```

Tal vez ya has oído hablar de `xgboost`. Este es un algoritmo de otra librería y se usa a menudo en competencias de aprendizaje automático. No lo necesitarás en este curso, pero puedes leer más aquí:

<https://xgboost.readthedocs.io/en/latest/>

<https://www.datacamp.com/community/tutorials/xgboost-in-python>

Algoritmos de aprendizaje no supervisado: clustering

En general, el aprendizaje no supervisado se trata de encontrar una similitud entre los objetos. Una vez que el modelo la ha detectado, puedes agrupar los objetos, es decir, resolver un **problema de agrupamiento**. Es posible que también encuentres similitudes entre las características. Será el **problema de reducción de dimensionalidad**.

Echemos un vistazo más de cerca al *clustering*. Es muy común en los negocios. Por ejemplo, el *clustering* se utiliza:

- Para la segmentación de clientes
- Para la segmentación de productos
- Para el modelado de temas

¿Qué tiene que ver todo esto con la distancia?

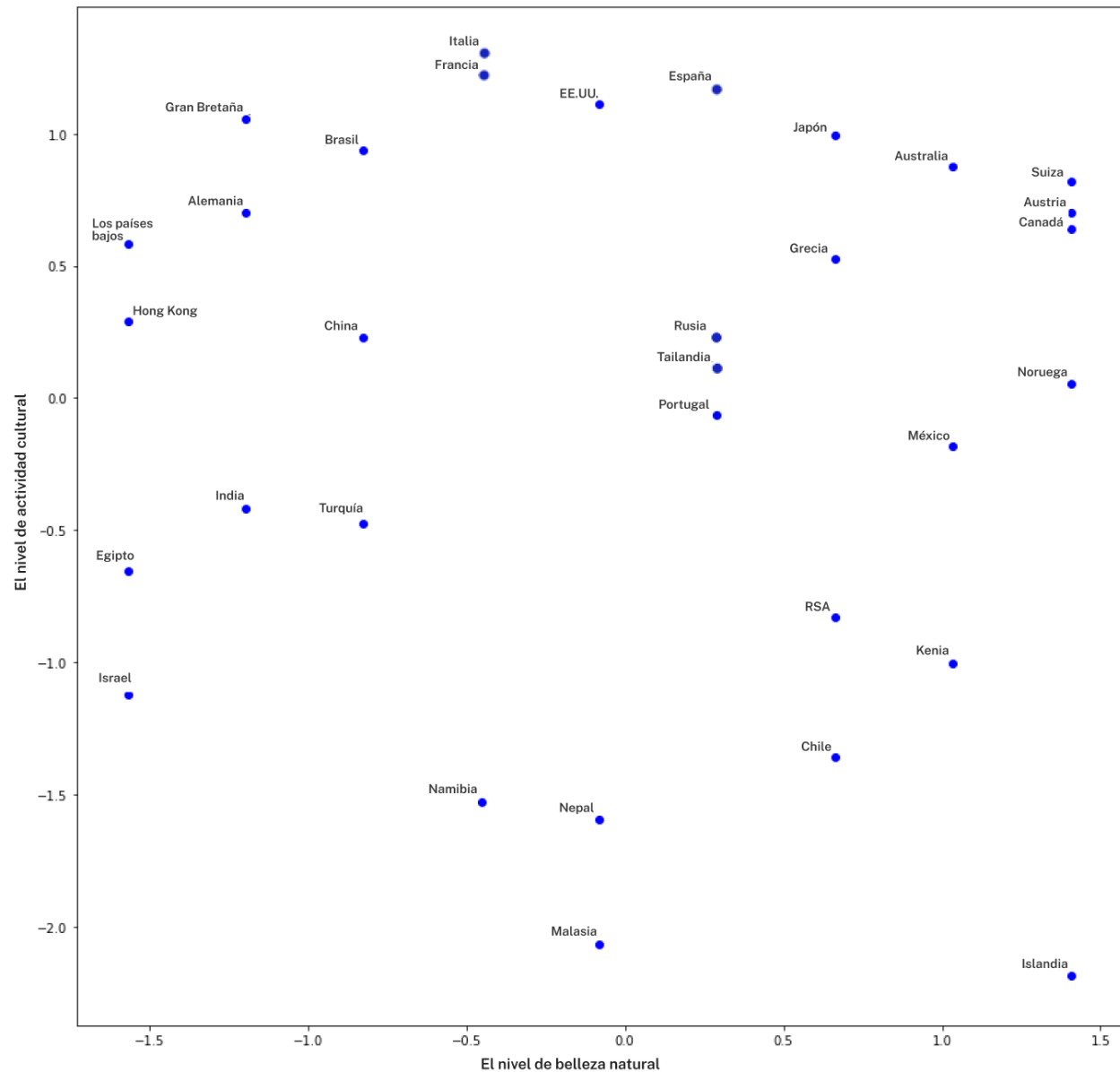
Los objetos similares comparten características similares. En otras palabras, la "distancia" entre ellos es pequeña. Acabamos de sustituir el concepto de "similaridad" con el de "distancia", con el que es más fácil de trabajar a nivel intuitivo.

El primer paso en un problema de *clustering* es determinar la distancia entre los objetos, es decir, describir su proximidad en números. Después puedes agrupar los países en función de su proximidad. Definiremos la función para la distancia entre objetos así:

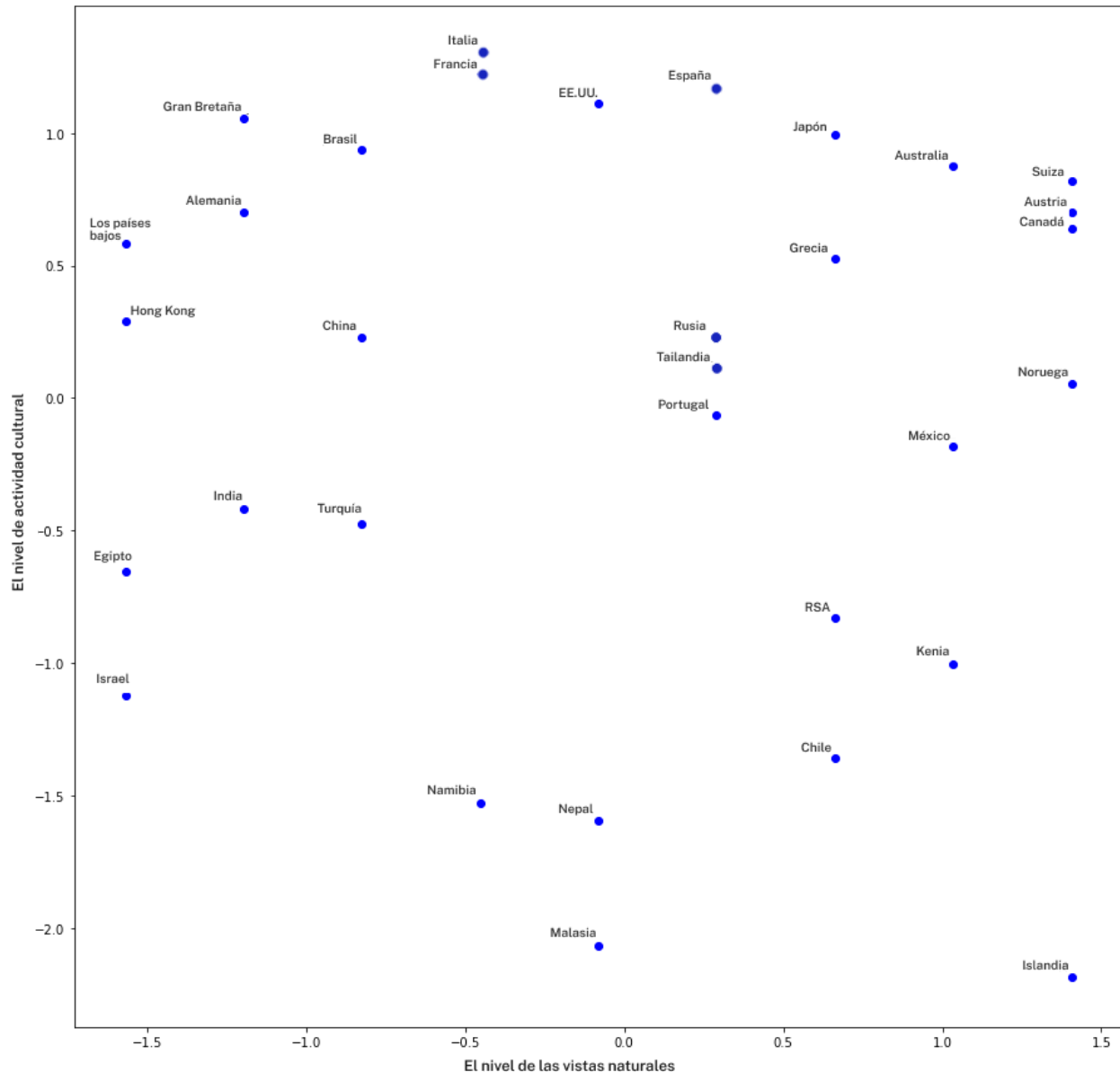
$$\rho(x_1, x_2)$$

Dejemos a cada país con solo dos características. ¿Cómo podemos definir la distancia entre dos objetos en función de los datos de dos características? Podemos trazar un

gráfico con las puntuaciones de la naturaleza en el eje X y el número de instituciones culturales a lo largo del eje Y. Terminaremos con algo así:



Pero sería buena idea unificar las escalas de características primero. Aquí los métodos de **normalización** y **estandarización** que ya conoces serán prácticos. Recuerda que la normalización asignará valores entre 0 y 1 a las características. La estandarización transformará los valores de modo que formen una distribución normal estándar: la media será 0 y su desviación estándar será 1. La visualización de características de datos estandarizados podría verse así:



¿Cómo podemos encontrar la distancia si conocemos los valores de característica de dos objetos cualesquiera? Aplica la fórmula de distancia estándar (a veces llamada distancia euclidiana):

$$\rho(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1_i} - x_{2_i})^2}$$

Aquí n es el número de dimensiones (países). Entonces, si solo tenemos dos características, la distancia entre los países será definida así:

$$\rho(x_1, x_2) = \sqrt{(x_{1_1} - x_{2_1})^2 + (x_{1_2} - x_{2_2})^2}$$

Clustering K-Means y clustering jerárquico aglomerativo

Los algoritmos de *clustering* más populares son:

- **K-means**
- **Clustering jerárquico aglomerativo**

El primer método es rápido y claro, aunque sensible al número de clústeres que definas. El segundo es más complicado, pero no requiere establecer el número de clústeres con antelación y te permite visualizar la relación entre objetos. Veamos cada uno en orden.

K-means

K-means agrupa objetos paso por paso. El algoritmo se basa en la suposición que el número de clústeres (grupos) ya se conoce. Esta es una suposición bastante fuerte y encontrar el número óptimo de clústeres a menudo vale la pena para resolver tareas separadas.

Este es el algoritmo para *K-means*:

1. Tenemos clústeres K. El algoritmo toma sus centros uno a uno y coloca cada objeto en el clúster cuyo centro es el más cercano.
2. Los centros se corrigen (se mueven) hasta que la distancia media entre los objetos dentro de cada clúster y su centro se minimiza.
3. Cuando la distancia desde los objetos al centro para de disminuir o disminuye solo de manera insignificante, el algoritmo para y arregla la división, reconociéndolo como el óptimo.

Ahora que sabemos cómo funciona el algoritmo, ejecutémoslo en Python.

El algoritmo llamado `KMeans` se implementa en el módulo `sklearn.cluster`. Así se ve su sintaxis:

```
from sklearn.cluster import KMeans

# la estandarización de datos obligatoria antes de pasarlo al algoritmo
sc = StandardScaler()
X_sc = sc.fit_transform(X)

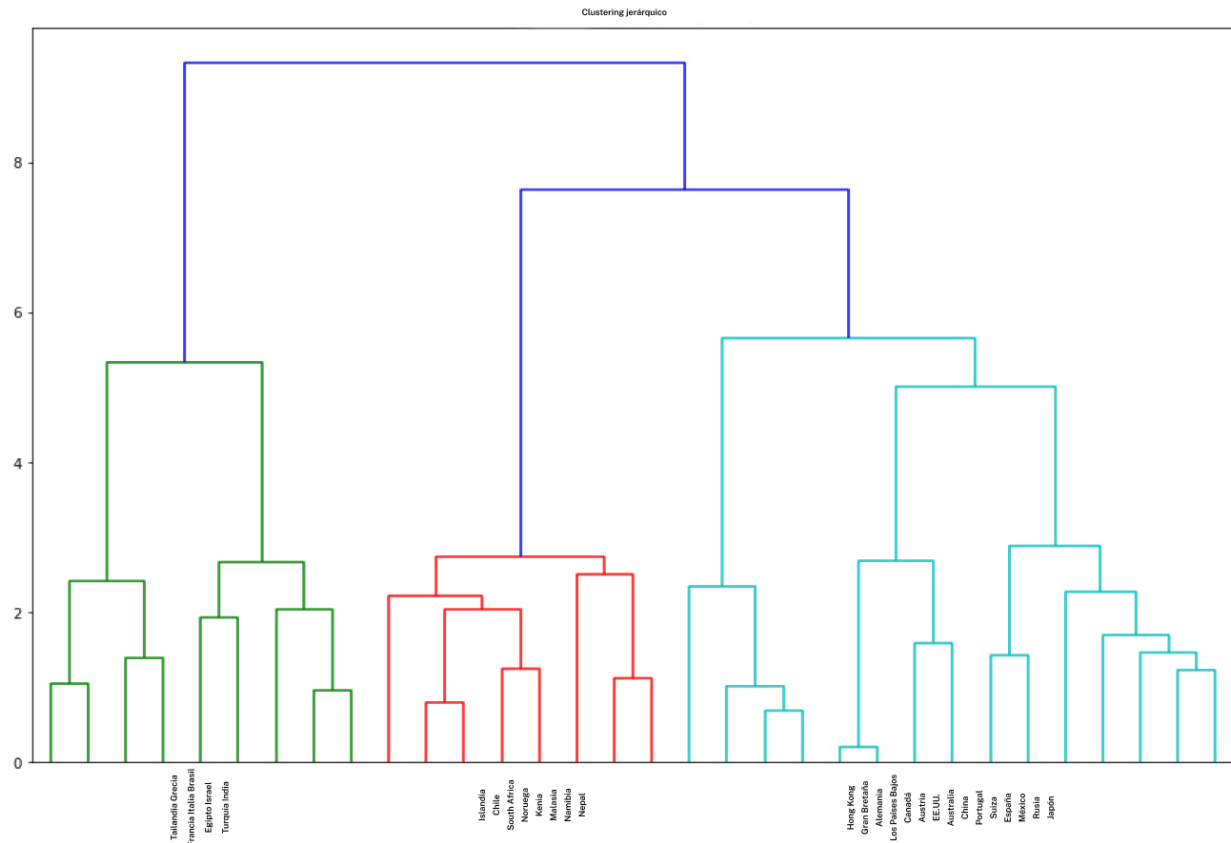
km = KMeans(n_clusters = 5) # establecer el número de clústeres
labels = km.fit_predict(X_sc) # aplicar el algoritmo a los datos
```

La variable `labels` almacena los índices de los grupos sugeridos por el algoritmo. El algoritmo asigna aleatoriamente un número a cada grupo, así que no sirve de nada intentar identificar la lógica: el grupo `2` no es necesariamente más cercano al grupo `3` que al grupo `1`. Lo más importante aquí es que los objetos con el mismo índice se refieren al mismo clúster.

Clustering jerárquico aglomerativo

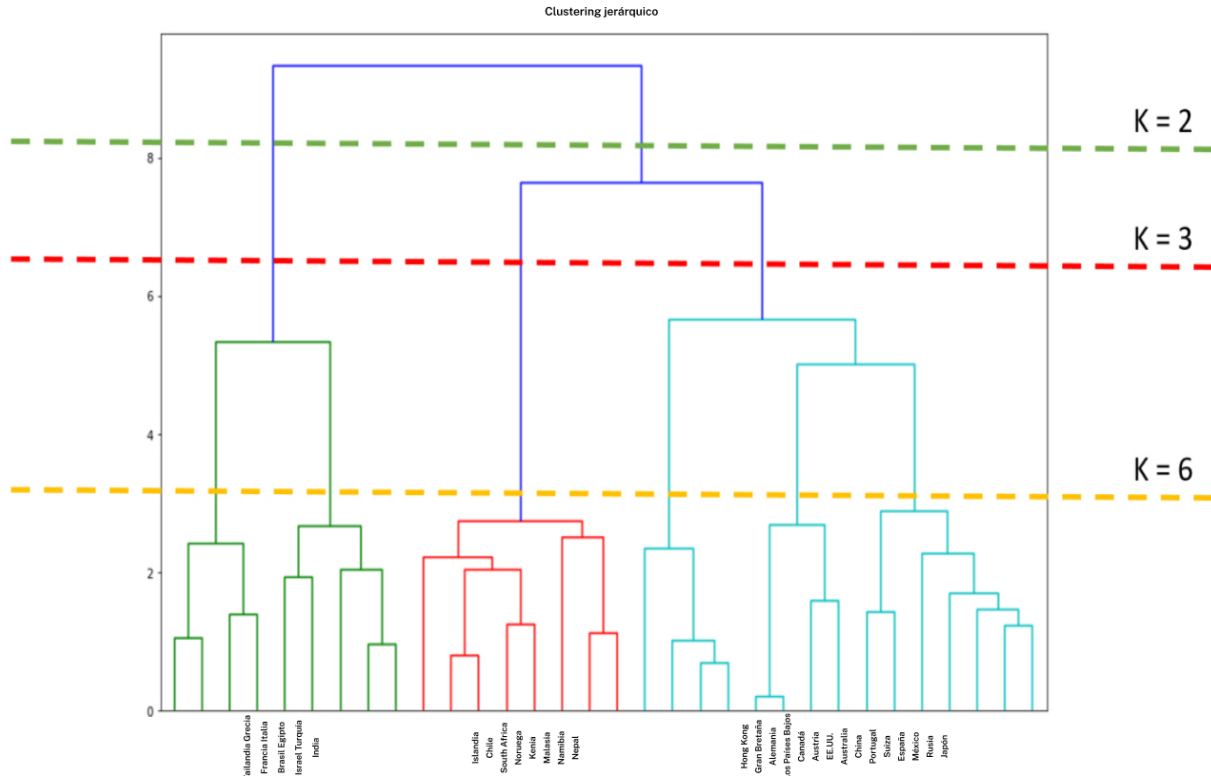
El quid del *clustering* jerárquico es sencillo. Cuando definimos la función de distancia, podemos calcular la matriz de distancias entre todos los objetos. Sus celdas contendrán la distancia entre parejas de objetos. Además, tendrá en cuenta todas las características de los objetos, no solo dos de ellas.

Podemos utilizar una matriz objeto-característica para juntar clústeres cercanos de manera uniforme. Es imposible estimar la proximidad de países separados o detectar rápidamente los países más cercanos o lejanos solo con mirar. Pero la distancia entre los objetos y el propio *clustering* jerárquico aglomerativo se puede visualizar con unos gráficos especiales llamados **dendrogramas**.



La distancia entre clústeres va en el eje Y y los objetos en el eje X. Cada enlace horizontal corresponde a la distancia entre los objetos que se unen. Si nos movemos desde el fondo a la parte de arriba del gráfico, primero veremos enlaces que unen objetos separados. Los siguen enlaces que unen objetos con grupos o grupos con grupos. Si no paras este proceso deliberadamente, acabará cuando quede un clúster gigante. Así es cómo un **clustering jerárquico aglomerativo** funciona de verdad. Este método de *clustering* es realmente intuitivo. En cada paso, el algoritmo amplía los clústeres añadiendo a los colindantes. Por eso se llama aglomerativo: convierte los clústeres en aglomeraciones. (Así también llamamos a un grupo de áreas residenciales vecinas). Lo importante es definir el momento en el que necesitamos parar el proceso de unión.

Puedes utilizar dendrogramas para estimar visualmente el número necesario de clústeres. También podemos hacernos una idea de la distancia a partir de la cual dejamos de unir objetos. Este es un ejemplo de cómo podemos seleccionar diferentes umbrales de distancia para un gran número de objetos:



El valor K es el número de líneas cruzadas por la línea discontinua.

Tracemos un dendrograma en Python. Primero importaremos las clases `linkage()` y `dendrogram()` del modelo de *clustering* desde el módulo de *clustering* `hierarchy`:

```
from scipy.cluster.hierarchy import dendrogram, linkage
```

Después estandarizaremos los datos y pasaremos la tabla resultante como un parámetro a la función `linkage()`. Para hacer nuestro gráfico más representativo, pasaremos `'ward'` al parámetro `method`:

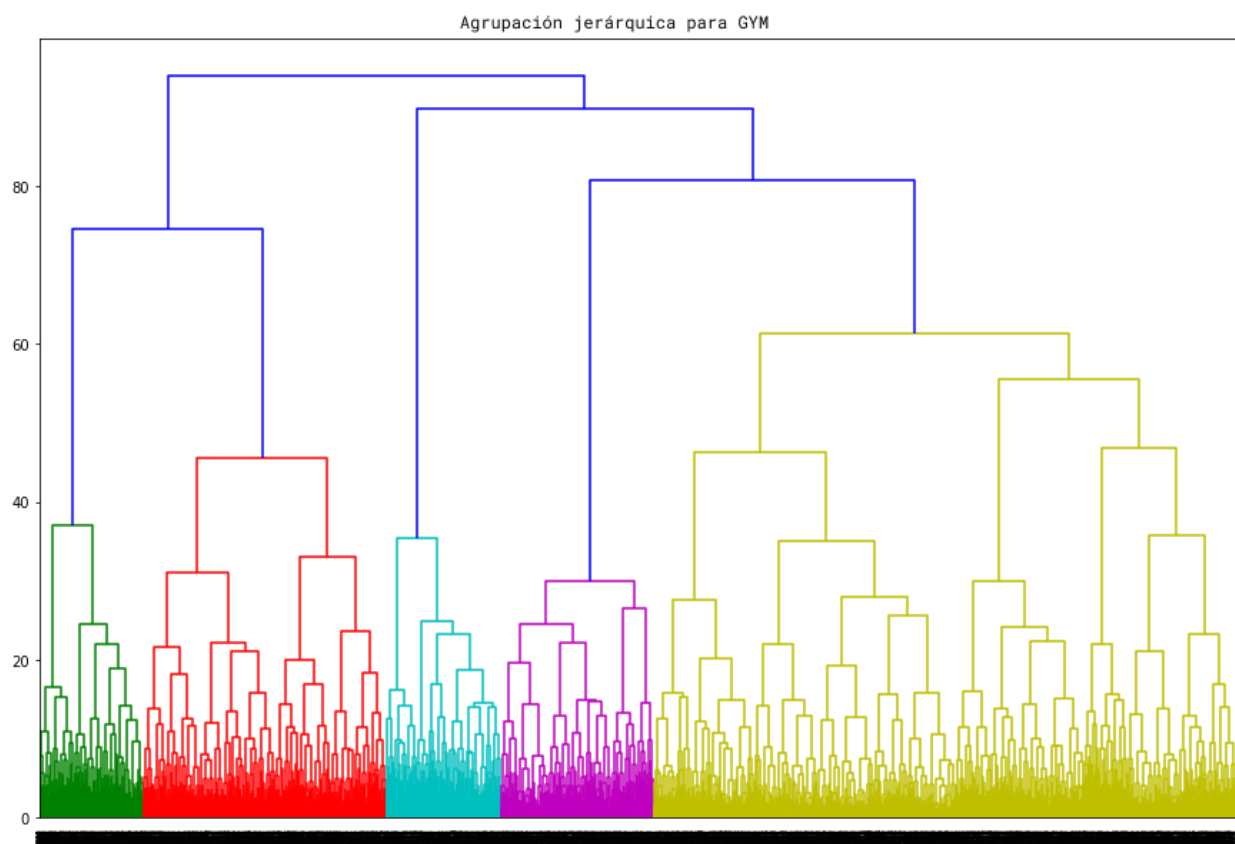
```
# estandarización obligatoria de los datos antes de pasarlo al
sc = StandardScaler()
X_sc = sc.fit_transform(X)

linked = linkage(X_sc, method = 'ward')
```

La variable `linked` almacena la tabla con los haces de objetos vinculados. Puede visualizarse como un dendrograma:

```
plt.figure(figsize=(15, 10))
dendrogram(linked, orientation='top')
plt.title('Agrupación jerárquica para GYM')
plt.show()
```

Obtendremos el siguiente gráfico:



El número óptimo de clústeres sugerido (5) corresponde a los cinco colores diferentes del gráfico.

La complejidad del *clustering* aglomerativo tiene que ver con los cálculos que hace la máquina para trazar un dendrograma, más que con la disposición del algoritmo como tal. La computación de distancias pareadas puede llevar mucho tiempo, por lo que al resolver un problema de *clustering*, puedes trazar un dendrograma en un subconjunto

aleatorio, estimar el número óptimo de clústeres y lanzar el algoritmo *K-means* más rápido.

Métricas para los problemas de aprendizaje no supervisado

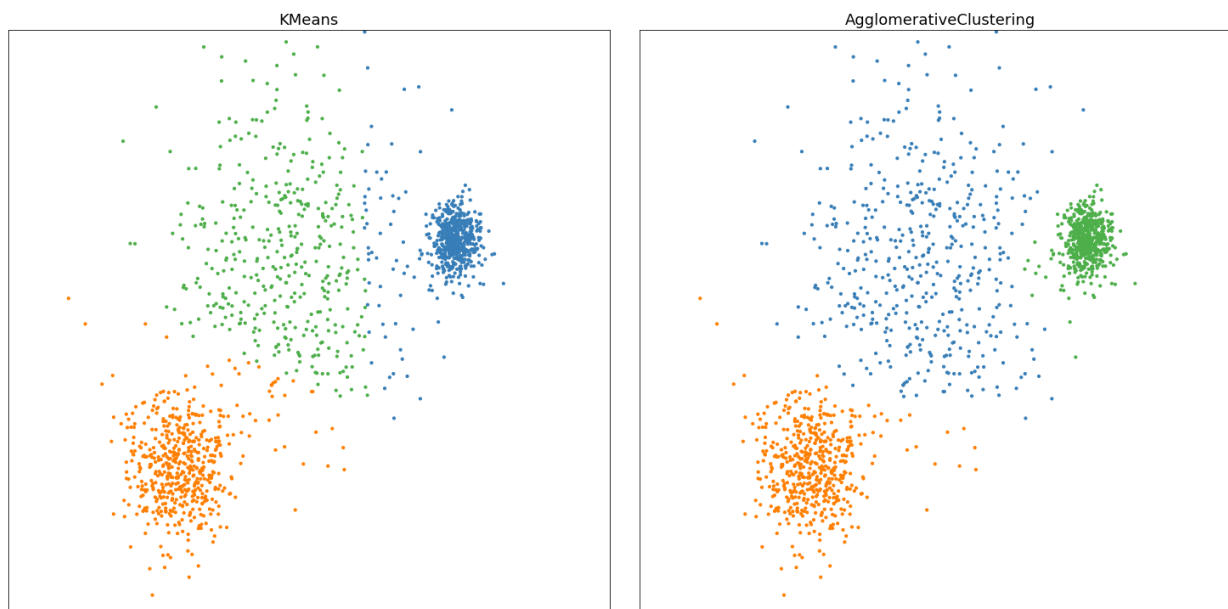
Un dato curioso: hay un *teorema de imposibilidad para el clustering* formulado por Jon Kleinberg, según el cual no existe tal cosa como un algoritmo óptimo para el *clustering*. Esto significa que, dado que no hay etiquetado, no hay una forma correcta de evaluar los resultados del desempeño de un algoritmo. Todo los enfoques para determinar si los objetos se separan bien se basan en opiniones expertas o evaluaciones heurísticas (por ensayo y error). No obstante, en los negocios, las tareas de *clustering* son muy comunes y debe haber una forma de evaluarlas.

De hecho, todo era sencillo cuando tratábamos con métricas para los problemas con datos etiquetados. Teníamos predicciones, teníamos los valores reales de la variable objetivo y teníamos expertos y expertas en aprendizaje automático para mostrarnos de varias formas en que diferían los dos.

Pero, ¿cómo podemos decir que nuestro modelo es el correcto cuando no tenemos respuestas correctas? ¿Cómo podemos evaluar la calidad de los algoritmos que agrupan objetos de acuerdo con la similitud?

La respuesta yace en la forma en la que establecemos la tarea. Si nos movemos desde la parte inferior hasta la parte superior del gráfico, veremos enlaces que unen objetos separados, seguidos de enlaces que unen objetos con grupos o grupos con grupos.

Tú has dividido a los clientes en grupos utilizando el *clustering K-means* y el *clustering* jerárquico. ¿Cómo podemos determinar qué algoritmo hizo un mejor trabajo? Supón que tienes solo dos características y el *clustering* final se ve así:



Vas a ver cuán "distinguibles" son las clases. ¿Hay alguna diferencia visual entre los objetos de diferentes clústeres en estas imágenes?

Para conseguir una solución más objetiva, necesitamos los conceptos de **distancias intraclúster** e **interclúster**. Como era de esperar, la distancia intraclúster es la distancia entre objetos dentro de un clúster mientras que la distancia entre grupos es la distancia entre objetos de diferentes clústeres. Cuanto mayor sea la diferencia entre las distancias intraclúster e interclúster, mejor será el *clustering*.

Hay una métrica popular llamada **valor de la silueta**. Hay otras métricas por ahí, pero son parecidas. El valor de la silueta muestra hasta qué punto un objeto de un clúster es similar a su clúster, en vez de otro. Aquí tienes la fórmula:

$$Silhouette = \frac{1}{n} * \sum_{c_k \in C} \sum_{x_i \in c_k} \frac{b(x_i, c_k) - a(x_i, c_k)}{\max[a(x_i, c_k), b(x_i, c_k)]}$$

donde n es el número de observaciones,

c_k es un clúster específico,

x_i es un objeto específico de c_k ,

$a(x_i, c_k)$ es la distancia media de x_i a los otros objetos en el clúster (una medida de la **densidad** del clúster),

$b(x_i, c_k)$ es la distancia media de x_i a objetos de otro clúster (una medida de la **divisibilidad** del clúster).

La sintaxis de este método es así:

```
from sklearn.metrics import silhouette_score

silhouette_score(x_sc, labels)
```

Los datos de entrada son la matriz normalizada o estandarizada y una lista de etiquetas predichas por nuestro algoritmo de *clustering*.

El valor de la silueta toma valores de -1 a 1. Cuanto más cercano a 1, mejor será el *clustering*.